

UNIVERSITÀ POLITECNICA DELLE MARCHE

CORSO DI LAUREA MAGISTRALE IN INGEGNERIA
INFORMATICA E DELL'AUTOMAZIONE

Progetto Zero Trust Architecture 2026

*Protezione Adattiva di Database e API con Verifica Continua,
Machine Learning e Micro-Segmentazione*

Candidati:

Andrea Flaiani (Matr. 1126928)
Andrea Altieri (Matr. 1128865)
Niccolò de Pascali (Matr. 1123958)
Matteo Risolo (Matr. 1122743)
Simone Murazzo (Matr. 1113295)

Docente:

Prof. Luca Spalazzi

Indice

1	Introduzione e obiettivi	6
1.1	La protezione dei dati aziendali e delle risorse critiche	6
1.2	La filosofia Zero Trust: “never Trust, always Verify”	6
1.3	Il caso di studio: l'infrastruttura ospedaliera	7
1.4	Obiettivi del progetto	7
1.5	Struttura della relazione	8
2	Architettura di sistema e microsegmentazione	9
2.1	Modellazione delle minacce e analisi del rischio	9
2.2	Le quattro reti segregate	10
2.3	Il choke-point obbligato: envoy proxy	12
2.4	La tupla multidimensionale Zero Trust (ZTA 6D)	13
2.5	Orchestrazione e Isolamento dei Container	14
2.6	Il server database di risorsa (MongoDB)	14
2.7	Le Web API e il portale client di simulazione	15
3	Implementazione pratica e configurazione	17
3.1	La microsegmentazione e il deploy con docker-compose	17
3.2	La configurazione di Envoy Proxy come PEP	17
3.3	Formalizzazione del Controllo degli Accessi: Modello ABAC e Matrice Dinamica	20
3.4	Il motore decisionale OPA (PDP) e l'integrazione Splunk MLTK	22
3.5	Regole di network protection e tracciamento su nftables	24
3.6	Autenticazione crittografica: mTLS, TPM e fingerprinting JA3	26
3.6.1	Mutua autenticazione TLS (mTLS)	26
3.6.2	Attestazione hardware TPM	26
3.6.3	Fingerprinting software JA3	28
3.7	Rilevamento delle intrusioni di rete: Snort NIDS	28
3.8	Aggregazione centralizzata dei log: Fluent-Bit	30
3.9	Il modello predittivo di Machine Learning: Splunk MLTK e Gradient Boosting	30
3.9.1	Dall'approccio statico alla valutazione comportamentale continua	31
3.9.2	Generazione del dataset e modello matematico	31
3.9.3	Selezione dell'algoritmo: Gradient Boosting vs Random Forest	33
3.9.4	Addestramento e Regressione in tempo reale	33
4	Simulazione degli scenari e validazione	35
4.1	Scenario 1: Tentativo di connessione senza certificato client	35
4.2	Scenario 2: Tentativo di accesso con certificato non firmato dalla CA	36
4.3	Scenario 3: Tentativo di bypass L4 e rilevamento attivo	37
4.4	Scenario 4: Accesso legittimo da rete interna: Alice	39
4.5	Scenario 5: Blocco al login per mancanza di TPM: Bob	41
4.6	Scenario 6: Accesso legittimo da rete esterna: Charlie	43
4.7	Scenario 7: Accesso ai dati sensibili: Alice (utente interno)	44

4.8	Scenario 8: Accesso ai dati sensibili: Charlie (utente remoto)	45
4.9	Scenario 9: Protezione dell'accesso TCP diretto	46
4.10	Scenario 10: Protezione dai payload dannosi a livello applicativo (L7) .	48
4.10.1	Simulazione di compromissione del dispositivo e blocco comportamentale	49
4.11	Scenario 11: Rilevamento di scansione di rete attiva (Nmap)	53
4.12	Scenario 12: Audit delle connessioni native MongoDB	54
4.13	Scenario 13: Rilevamento di attacchi volumetrici (DoS)	54
4.14	Scenario 14: Rilevamento di attacchi di downgrade (traffico in chiaro) .	55
4.15	Monitoraggio e Analisi in tempo reale: Splunk Dashboards	56
5	Conclusioni e sviluppi futuri	58
5.1	Obiettivi raggiunti	58
5.2	Sviluppi futuri	58

Elenco delle figure

1	Topologia di rete dell'infrastruttura Zero Trust con flussi logici (inclusa l'ispezione diretta Envoy-MongoDB) e canali di telemetria.	11
2	Diagramma dei flussi informativi e dei componenti della Zero Trust Architecture.	13
3	Schermata del portale di login protetto.	35
4	Schermata del pannello di ricerca di Splunk che mostra il log di blocco [NFT-BLOCK] indicizzato.	38
5	Mappatura logica dei flussi nello Scenario 3 (Tentativo di bypass e predisposizione architetturale per l'isolamento automatico).	39
6	Interfaccia del database pazienti visualizzata con successo da Alice dopo validazione TPM e rischio minimo.	40
7	Log di autorizzazione (ALLOW) per l'accesso di Alice indicizzato in Splunk, comprensivo del risk score comportamentale calcolato (≈ 9.96).	40
8	Mappatura logica dei flussi nello Scenario 4 (Accesso legittimo di Alice).	41
9	Schermata di errore 403 Forbidden mostrata a Bob per mancata attestazione hardware del chip TPM.	42
10	Log di diniego (DENY) per l'accesso di Bob indicizzato in Splunk, evidenziante l'assenza della firma hardware TPM (<code>tpm_present: false</code>) e l'assegnazione automatica del rischio massimo.	42
11	Mappatura logica dei flussi nello Scenario 5 (Blocco al login di Bob per mancanza di TPM).	43
12	Dashboard di Charlie dopo l'autenticazione riuscita da rete esterna.	44
13	Log di autorizzazione (ALLOW) per Charlie: il verdetto è positivo nonostante la rete esterna (<code>network_internal: false</code>), grazie al basso <i>risk_score</i> ($17.7 \leq 26$).	44
14	Mappatura logica dei flussi nello Scenario 6 (Accesso autorizzato per rischio contenuto).	44
15	Visualizzazione autorizzata del Fascicolo Sanitario Elettronico contenente dati sensibili da parte di Alice.	45
16	Pagina di blocco 403 Forbidden visualizzata da Charlie nel tentativo di accedere ai dati sensibili da remoto.	46
17	Log di blocco (DENY) per l'accesso ai dati sensibili di Charlie: il sistema applica il blocco DPI (<code>17_dpi_block: true</code>) a causa della rete esterna.	46
18	Interfaccia di blocco visualizzata da Alice a seguito del tentativo di esecuzione di metodo DELETE.	48
19	Log dell'evento di blocco indicizzato in Splunk: evidenza dell'operazione DELETE respinta.	49
20	Log di decisione DENY indicizzato in Splunk: l'accesso diretto viene bloccato nonostante la validità crittografica, a causa del rischio comportamentale estremo ($95.49 > 38$).	52
21	Mappatura logica dei flussi nello Scenario 10 (Blocco predittivo di una connessione per anomalie comportamentali, nonostante credenziali mTLS/TPM valide).	53

22	Log di alert Snort (ZTA ALERT: Rilevata Scansione di Rete) centralizzato su Splunk, relativo allo scan Nmap eseguito da Charlie.	54
23	Log di audit per connessione nativa a MongoDB non autorizzata.	54
24	Log di alert Snort relativo all'anomalia volumetrica (DoS) verso il proxy Envoy.	55
25	Log di alert Snort relativo al rilevamento di traffico HTTP in chiaro su porta protetta (Downgrade).	55
26	Dashboard ZTA - Security Executive: Panoramica e statistiche sul traffico.	56
27	Dashboard ZTA - Incident Response: Monitoraggio minacce e blocchi attivi.	57
28	Dashboard ZTA - ML & Continuous Trust Analytics: Analisi predittiva.	57

Elenco dei listati

1	Definizione delle quattro reti segregate nel file docker-compose.yaml. . .	10
2	Definizione delle quattro reti segregate nel file docker-compose.yaml. . .	17
3	Estratto della configurazione del listener MongoDB in envoy.yaml. . . .	19
4	Configurazione del filtro HTTP Lua in envoy.yaml per la prima linea di difesa L7.	20
5	Struttura della query Splunk MLTK generata dinamicamente da OPA. . .	22
6	Regole di ispezione profonda L7 DPI per confidenzialità e integrità in rules.rego.	23
7	Regole di autorizzazione adattiva (Score-based) e short-circuiting (Criteria-based) in rules.rego.	24
8	Configurazione delle catene di filtering e NAT nello script di avvio del firewall.	25
9	Generazione dei certificati client con estensione TPM simulata.	27
10	Validazione dell'estensione TPM nel certificato client da parte di OPA. .	27
11	Regole di rilevamento avanzate (Volumetriche, Protocollari e Crittografiche) per Snort.	29
12	Query SPL per l'addestramento del modello Gradient Boosting su Splunk MLTK.	34
13	Blocco preventivo a livello di trasporto per assenza di certificato client. .	36
14	Rilevamento e blocco di un certificato estraneo alla Trust Chain ospedaliera.	36
15	Simulazione di attacco bypass diretto tramite curl dal terminale di Charlie. .	37
16	Dettaglio del log di blocco generato da nftables per il tentativo di bypass. .	38
17	Connessione TCP al database autorizzata per dispositivo conforme (Alice).	47
18	Tentativo di connessione TCP abortito per dispositivo non conforme (Bob). .	47
19	Estratto della logica di arricchimento comportamentale nel PIP.	50
20	Simulazione di accesso diretto al database MongoDB dal dispositivo compromesso di Alice.	51
21	Esecuzione di un service scan per mappare la superficie di attacco. . .	53
22	Tentativo di connessione TCP diretta al database MongoDB.	54
23	Generazione di traffico aggressivo per simulazione DoS.	54
24	Simulazione di richiesta HTTP in chiaro su porta protetta.	55

CAPITOLO 1

Introduzione e obiettivi

In questo primo capitolo viene presentato il contesto applicativo e metodologico alla base del progetto, definendo le sfide di sicurezza tipiche della gestione di database distribuiti e illustrando i principi teorici del paradigma Zero Trust. Si introduce quindi il caso di studio ospedaliero scelto come dominio di riferimento e si delineano gli obiettivi e la struttura complessiva della relazione.

1.1 La protezione dei dati aziendali e delle risorse critiche

Le infrastrutture IT aziendali rappresentano uno dei bersagli più ambiti e critici per gli attori malevoli a livello globale. La digitalizzazione dei servizi e la transizione verso sistemi in cloud hanno introdotto enormi benefici in termini di efficienza operativa, tra cui l'accesso condiviso a risorse sensibili, l'automazione dei flussi e il monitoraggio IoT di asset industriali. Tuttavia, questa evoluzione ha contemporaneamente esteso a dismisura la superficie d'attacco delle organizzazioni.

La criticità di questo scenario è dettata da molteplici fattori. In primo luogo, vi è la natura estremamente riservata delle informazioni trattate all'interno dei database, che includono dati finanziari, proprietà intellettuali, credenziali e profili di utenti. La perdita di riservatezza o l'alterazione di tali dati viola gravemente le principali normative vigenti sulla protezione dei dati personali e sulla sicurezza delle reti a livello internazionale. In secondo luogo, va considerato l'impatto sulla **business continuity**: a differenza dei semplici disservizi temporanei, un attacco informatico di tipo ransomware che cripta o altera i database operativi può causare il blocco completo dei servizi, enormi perdite finanziarie e danni reputazionali incalcolabili. Infine, l'eterogeneità degli accessi complica ulteriormente il quadro: gli operatori e gli amministratori devono poter consultare i dati sia dai terminali interni alla rete aziendale, come le workstation d'ufficio, sia da remoto, ad esempio in regime di smart working, introducendo vettori di minaccia esterni non controllati.

1.2 La filosofia Zero Trust: “never Trust, always Verify”

La sicurezza perimetrale classica, basata sul modello a “castello”, in cui un firewall protegge una rete interna considerata intrinsecamente sicura da una rete esterna ostile, si è dimostrata obsoleta e inefficace. Una volta che un attaccante riesce a penetrare il perimetro, tramite phishing, exploit o **insider threat**, ottiene totale libertà di movimento laterale.

Il paradigma **Zero Trust Architecture** (ZTA), codificato dal NIST nel documento speciale *SP 800-207*, supera questa concezione abbattendo il concetto di fiducia implicita basata sulla localizzazione di rete. I cardini della filosofia ZTA sono tre:

1. **Mai fidarsi, verificare sempre (Never Trust, Always Verify)**: ogni singola richiesta di accesso ad una risorsa, indipendentemente dalla rete da cui proviene, sia essa interna o esterna, deve essere autenticata, autorizzata e validata crittograficamente prima dell'inoltro.

2. **Accesso con privilegi minimi (Least Privilege):** l'accesso viene concesso in modo estremamente granulare e ristretto solo alle risorse necessarie per lo svolgimento della specifica attività, ad esempio limitando la scrittura e la cancellazione di record sensibili ai soli operatori autorizzati.
3. **Presumere la violazione (Assume Breach):** progettare il sistema assumendo che alcuni segmenti siano già compromessi. Ciò impone l'adozione di micro-segmentazione di rete per limitare il raggio d'azione dell'attaccante, crittografia end-to-end per proteggere i dati in transito e sistemi di monitoraggio continuo del traffico.

1.3 Il caso di studio: l'infrastruttura ospedaliera

Il dominio applicativo scelto per la validazione dell'architettura è quello di una **struttura ospedaliera**, un contesto in cui la criticità dei dati trattati e l'eterogeneità degli accessi rendono particolarmente evidente la necessità di un approccio Zero Trust.

Il sistema gestisce un database MongoDB contenente un esempio di informazioni sanitarie altamente sensibili, come anagrafiche dei pazienti, cartelle cliniche e dati diagnostici. L'accesso a queste risorse è richiesto da tre categorie di operatori con profili di rischio differenti:

- **Alice:** personale medico interno all'ospedale, dotato di workstation aziendale con chip **TPM** hardware e collegato dalla rete locale (10.0.1.0/24). Rappresenta l'utente a rischio minimo.
- **Bob:** operatore interno che utilizza un dispositivo personale privo di attestazione hardware TPM. Pur collegandosi dalla rete aziendale, la mancanza di garanzie sull'integrità del dispositivo lo sottopone a soglie di rischio più restrittive. Visto da un altro punto di vista, Bob rappresenta un esempio di **insider threat** potenziale, per esempio un utente che ha rubato le credenziali ma non possiede TPM.
- **Charlie:** professionista in regime di smart working, dotato di certificato e chip TPM ma collegato da una rete esterna non controllata (192.168.100.0/24). La provenienza da una rete non fidata impone politiche di tolleranza al rischio più restrittive.

Questo scenario consente di dimostrare concretamente come l'architettura adatti dinamicamente le proprie soglie di sicurezza in funzione del contesto multidimensionale di ciascun accesso, combinando l'identità dell'utente, la postura del dispositivo e la localizzazione di rete.

1.4 Obiettivi del progetto

Il presente progetto si pone l'obiettivo di realizzare un'architettura Zero Trust completa, funzionante ed integrata per la protezione di un database MongoDB aziendale. Il sistema è progettato per operare una valutazione continua e adattiva del rischio (*Continuous Authorization*) basata sul contesto di rete, sul comportamento dell'utente e sulla postura di sicurezza del dispositivo, inclusa rigorosamente l'attestazione hardware tramite modulo TPM.

Per raggiungere questo scopo, vengono impiegate diverse tecnologie integrate. In primo luogo, envoy opera come **Policy Enforcement Point** (PEP), intercettando ogni singola richiesta HTTP o MongoDB in modo sicuro mediante crittografia mutua TLS (*mTLS*), eseguendo il fingerprinting software (*JA3*) e l'ispezione profonda dei pacchetti a livello applicativo (*L7 DPI*). In secondo luogo, open policy agent (OPA) funge da **Policy Decision Point** (PDP) decentralizzato, valutando le richieste in tempo reale sulla base delle sei dimensioni Zero Trust (ZTA 6D), rappresentate dai parametri *u* (user), *d* (device), *s* (software), *n* (network), *a* (action), *r* (resource).

L'analisi comportamentale è affidata a splunk Machine Learning Toolkit (MLTK), interpellato sincronicamente da OPA per calcolare un punteggio di rischio dinamico tramite modelli predittivi basati sull'algoritmo **Gradient Boosting**. Parallelamente, splunk SIEM si occupa dell'ingestion asincrona dei log per il monitoraggio e la correlazione storica degli eventi. Infine, a livello di rete, nftables e snort (NIDS) garantiscono rispettivamente la microsegmentazione del traffico a livello L3/L4 e il rilevamento tempestivo di tentativi di movimento laterale o bypass del proxy.

1.5 Struttura della relazione

Il documento è organizzato come segue: il Capitolo 2 illustra l'architettura complessiva della topologia di rete a quattro zone. Il Capitolo 3 scende nel dettaglio dell'implementazione tecnica di ciascun componente di sicurezza, descrivendo il funzionamento di envoy, OPA, splunk, snort e nftables. Il Capitolo 4 descrive la validazione del sistema attraverso la simulazione di scenari d'attacco ed accessi legittimi. Infine, il Capitolo 5 riassume i risultati e propone futuri sviluppi progettuali.

CAPITOLO 2

Architettura di sistema e microsegmentazione

In questo capitolo viene analizzata l'architettura logica e fisica dell'infrastruttura, descrivendo la segmentazione di rete a quattro zone e il ruolo di envoy come unico punto di transito autorizzato per il traffico dati.

2.1 Modellazione delle minacce e analisi del rischio

In conformità con la rigorosa metodologia di analisi del rischio e *Threat Modeling* tratta nel corso di *Advanced Cybersecurity*, l'architettura implementata adotta un approccio analitico e quantitativo per la protezione dei componenti di sistema. Il modello si articola sui seguenti concetti fondamentali:

- **Asset** (*O*): Rappresentano le risorse logiche e fisiche di valore che l'organizzazione deve proteggere. Nel dominio specifico, essi comprendono il database MongoDB (*Data at Rest*), i flussi di comunicazione verso le Web API (*Data in Transit*) e le cartelle cliniche dei pazienti. Ad ogni asset è associato un valore di **Impatto** (*I*), che quantifica il danno per il business in caso di violazione della triade CID (Confidenzialità, Integrità, Disponibilità). Trattandosi di dati sanitari sensibili, l'impatto potenziale è classificato come severo.
- **Vulnerabilità** (*V*): Costituiscono le debolezze intrinseche del sistema, dei protocolli o delle procedure operative. Nell'architettura in esame, lo spazio delle vulnerabilità include la potenziale compromissione delle credenziali utente, l'esposizione di terminali client sprovvisti di attestazione hardware (chip TPM), l'utilizzo di canali non cifrati all'interno della rete e le inevitabili anomalie comportamentali fisiologiche degli operatori.
- **Minacce** (*T*): Sono definite come agenti o eventi (esterni o interni) in grado di sfruttare in modo opportunistico una determinata vulnerabilità *V* per compromettere un asset *O*. Le minacce modellate per questo scenario includono tentativi di esfiltrazione dati, *privilege escalation*, attacchi *injection* e *lateral movement* tra i container.

Per mitigare la superficie di attacco complessiva, il sistema adotta un paradigma di **Difesa in Profondità** (*Defense in Depth*), distinguendo tassativamente due livelli di controlli di sicurezza:

1. **Meccanismi di Prevenzione**: Agiscono *ex ante* per inibire il verificarsi della minaccia, riducendone la probabilità di successo. Rientrano in questa barriera primaria l'autenticazione mTLS, le policy di microsegmentazione e le regole deterministiche di OPA applicate tramite i filtri Envoy.

2. **Meccanismi di Rilevamento:** Agiscono *ex post* (o in *near real-time*) per identificare le minacce che sono riuscite a bypassare il primo strato difensivo. Strumenti come Snort e le dashboard di correlazione in Splunk assolvono questa funzione, garantendo visibilità e riducendo il tempo di latenza (*dwell time*) dell'attaccante all'interno della rete.

Il fulcro innovativo del sistema risiede nella gestione dinamica del **Rischio** (R). Secondo la teoria classica, il rischio operativo è quantificato dalla funzione $R = L \times I$, dove L rappresenta la probabilità (*Likelihood*) che una minaccia T sfrutti con successo una vulnerabilità V . Nei sistemi tradizionali, L è una metrica statica; nella presente architettura Zero Trust, L diventa una variabile dinamica continua.

Il *Trust Score* elaborato in tempo reale dal modello MLTK di Splunk agisce come una stima inversamente proporzionale di L : al degradare della fiducia comportamentale dell'utente, la stima della probabilità di compromissione aumenta. Ad ogni richiesta, il Policy Decision Point (PDP) calcola il rischio attuale R e lo confronta con il **Rischio Accettabile** (R_a), ovvero la soglia massima di tolleranza definita dalle policy di business. Il sistema autorizza e instrada la transazione esclusivamente se è verificata la condizione formale $R \leq R_a$. In caso contrario, subentra un meccanismo di *short-circuiting*: la connessione viene immediatamente bloccata (a livello L7 tramite OPA o a livello L3/L4 tramite nftables), riconducendo forzatamente l'infrastruttura entro uno stato di sicurezza accettabile.

2.2 Le quattro reti segregate

La sicurezza dell'infrastruttura si basa sul principio della **microsegmentazione**, implementata definendo quattro reti virtuali isolate tramite Docker. Questa compartimentazione riduce drasticamente l'area di impatto di un'eventuale compromissione. La segmentazione fisica ed i relativi indirizzamenti IP delle quattro aree protette sono schematizzati visivamente nella Figura 1, mentre la loro definizione a livello di infrastruttura è dettagliata nella configurazione mostrata nel Listato 1.

```

●●● docker-compose.yaml
1 networks:
2   frontend-net:
3     driver: bridge
4     ipam:
5       config:
6         - subnet: 10.0.1.0/24
7   backend-net:
8     driver: bridge
9   control-plane-net:
10    driver: bridge
11   external-net:
12    driver: bridge
13    ipam:
14      config:
15        - subnet: 192.168.100.0/24

```

Listing 1: Definizione delle quattro reti segregate nel file docker-compose.yaml.

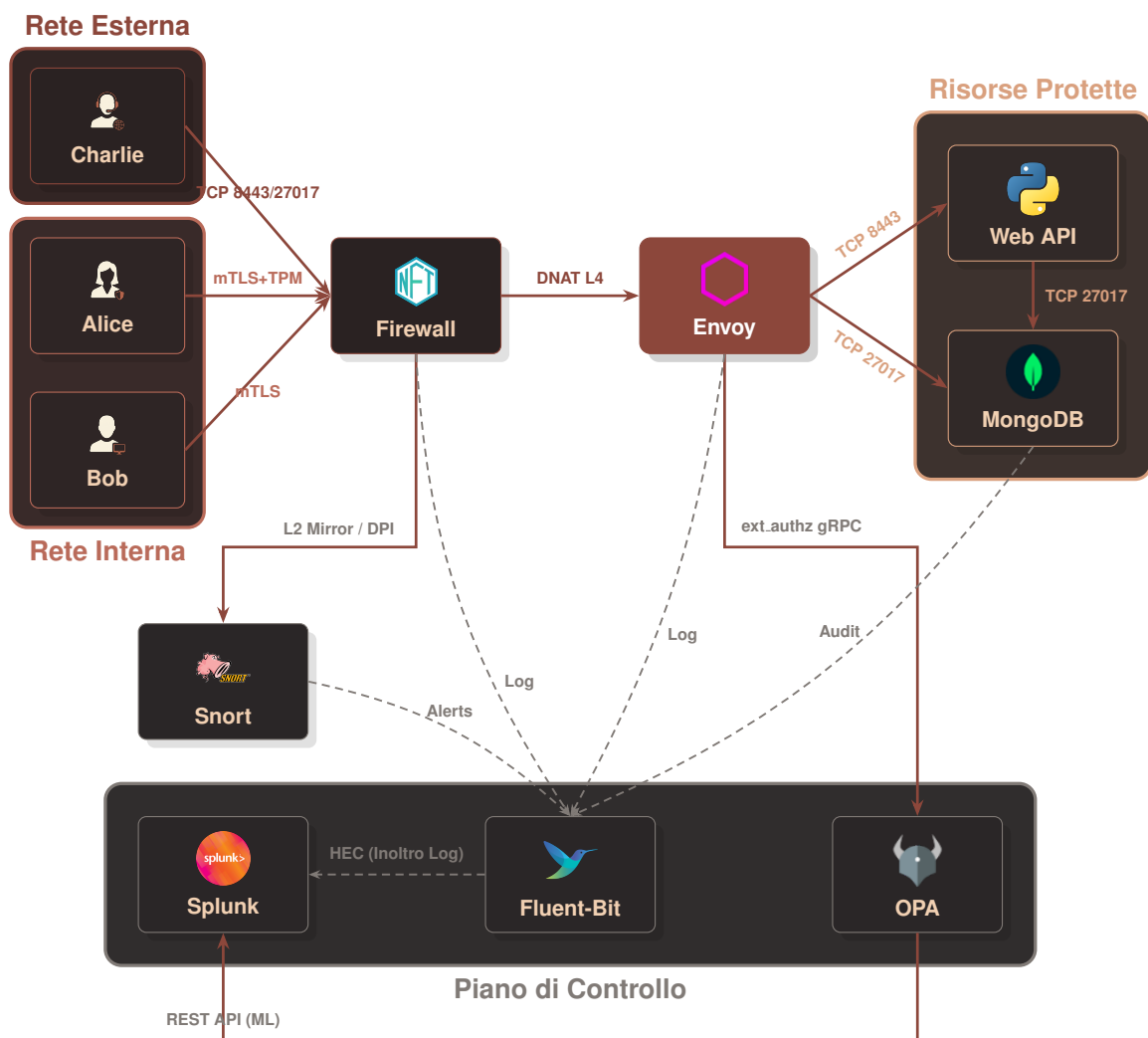


Figura 1: Topologia di rete dell'infrastruttura Zero Trust con flussi logici (inclusa l'ispezione diretta Envoy-MongoDB) e canali di telemetria.

La prima area è la **zona untrusted** (`external-net`), che rappresenta la rete esterna non controllata da cui provengono le richieste di utenti remoti. In questa zona opera, ad esempio, il client dell'operatore in smart working. Non vi è alcuna rotta di instradamento che colleghi direttamente questa zona al database o ai servizi interni: l'unico punto di contatto esposto è il listener di envoy, rigorosamente protetto da crittografia.

La seconda area è la **zona DMZ o frontend** (`frontend-net`), assimilabile alla rete locale fisica dell'azienda. Qui risiedono i client interni utilizzati dal personale d'ufficio e il firewall `nftables`. Anche in questo caso, i client del frontend non possiedono le rotte di rete dirette per raggiungere il backend: il firewall intercetta il traffico a livello L3/L4 e ne forza l'instradamento verso il proxy (envoy), unico attore preposto all'ispezione e all'autorizzazione delle richieste.

La terza area è il **secure core o backend** (`backend-net`), una rete altamente protetta e completamente isolata. All'interno di questa zona sono collocati il database `mongodb` e le Web API del servizio aziendale. Nessun client, sia esso esterno o interno, può avviare una connessione diretta con queste macchine: l'unico container autorizzato a inoltrare pacchetti nel backend è il proxy, che agisce da intermediario per le sole richieste esplicitamente validate dal motore decisionale. In merito alla scelta del database, le specifiche del progetto consentivano l'adozione di `mongodb` o, in alternativa, di una configurazione composta da `proxysql` e `mysql`. In questa architettura si è preferito utilizzare il database orientato ai documenti `mongodb` per agevolare l'ispezione granulare dei comandi BSON nativi direttamente a livello applicativo (*L7 DPI*), sfruttando il filtro di decodifica specifico di envoy (`envoy.filters.network.mongo.proxy`).

La quarta e ultima area è la **control plane o trust zone** (`control-plane-net`). Si tratta di una rete di gestione isolata in cui risiedono i sistemi decisionali e di monitoraggio, tra cui il motore delle regole OPA, il raccoglitore di log `fluent-bit` e il SIEM `splunk`. Questa zona comunica esclusivamente con il proxy per fornire in tempo reale i verdeti di validazione delle richieste e con tutti i container per la raccolta asincrona della telemetria in background.

2.3 Il choke-point obbligato: envoy proxy

Al centro dell'intera topologia si colloca envoy, che agisce come **Policy Enforcement Point** (PEP) e rappresenta il punto di strozzatura obbligato per tutto il traffico. Qualsiasi tentativo di comunicazione che cerchi di scavalcare questo passaggio viene intercettato e scartato a livello di rete dal firewall. Il posizionamento strategico del PEP e il flusso dei messaggi di autorizzazione verso gli altri moduli sono illustrati nella Figura 2.

Il funzionamento del proxy prevede la separazione dei flussi su porte distinte per gestire in modo sicuro i diversi protocolli. Sulla porta 8443 viene gestito il traffico HTTP cifrato per le Web API aziendali, mentre sulla porta 27017 transita il traffico TCP nativo diretto al database, anch'esso incapsulato in una connessione TLS protetta. Per ogni connessione, il proxy avvia una negoziazione crittografica pretendendo la presentazione di un certificato client valido (*mTLS*). Durante questa fase, viene calcolato l'hash della firma del software client per determinare il fingerprinting JA3.

Prima di inoltrare la richiesta verso il backend, il proxy sospende la sessione e interroga OPA tramite una chiamata gRPC. Solo dopo aver ricevuto un responso positivo dal PDP, il proxy provvede a stabilire la connessione con la risorsa richiesta nel backend, garantendo l'applicazione del paradigma Zero Trust per ogni singola transazione.

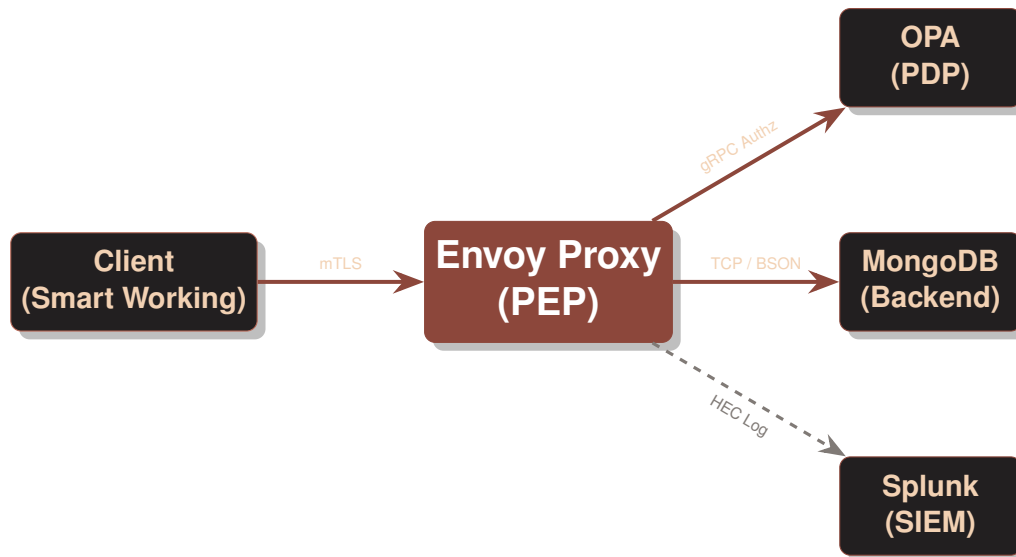


Figura 2: Diagramma dei flussi informativi e dei componenti della Zero Trust Architecture.

Questa separazione dei ruoli rispecchia una precisa stratificazione a livello della pila protocollare ISO/OSI. Mentre il firewall `nftables` opera a livello di rete e trasporto (L3/L4), intercettando e filtrando i pacchetti in base agli indirizzi IP e alle porte TCP/UDP, il proxy `envoy` lavora fino al livello applicativo (L7) e di sessione/presentazione (TLS), consentendo una validazione profonda e contestuale che non sarebbe possibile con un tradizionale filtro di rete a pacchetti.

2.4 La tupla multidimensionale Zero Trust (ZTA 6D)

Il nucleo del motore decisionale risiede nella formalizzazione matematica e logica del contesto di accesso attraverso una tupla a sei dimensioni, definita come:

$$T = (u, d, s, n, a, r) \quad (1)$$

Ciascuna dimensione mappa un attributo specifico estratto dinamicamente a livello L4-L7 e valutato in tempo reale per determinare l'affidabilità della richiesta:

1. **Utente** ($u \in U$): l'identità forte del soggetto richiedente, estratta dal campo `Subject` del certificato client X.509 presentato durante la fase di handshake mTLS (`DOWNSTREAM_PEER_SUBJECT`). Questa dimensione identifica univocamente gli operatori (*Alice, Bob, Charlie*).
2. **Dispositivo** ($d \in D$): lo stato di postura dell'hardware. Viene validato verificando la presenza di un'estensione X.509 personalizzata con Object Identifier (OID) specifico `1.3.6.1.4.1.9999.1`. Tale estensione attesta che la chiave privata del client sia custodita all'interno di un chip **TPM (Trusted Platform Module)** hardware o emulato, discriminando tra workstation sicure autorizzate e dispositivi non censiti.
3. **Software** ($s \in S$): il fingerprint crittografico della suite client, calcolato tramite l'algoritmo **JA3**. Questo consente a Envoy di identificare il software effettivo che avvia la connessione (es. un browser autorizzato vs uno script Python malevolo), prevenendo attacchi basati su spoofing dell'User-Agent.

4. **Contesto di Rete** ($n \in N$): la provenienza geografica o topologica del pacchetto, classificata in base all'indirizzo IP di origine (DOWNSTREAM_REMOTE_ADDRESS). Permette di distinguere le richieste provenienti dalla rete locale interna ospedaliera (10.0.1.0/24) da quelle provenienti dall'esterno (Smart Working tramite la subnet 192.168.100.0/24).
5. **Azione** ($a \in A$): l'operazione richiesta. Nel caso del protocollo HTTP corrisponde al metodo (es. GET, POST), mentre per le connessioni database MongoDB viene estratta a livello L7 dal filtro mongo_proxy (es. comandi di find, insert, update o delete).
6. **Risorsa** ($r \in R$): il target dell'azione, rappresentato dall'URI del servizio API HTTP richiesto o dal nome della collezione MongoDB (es. patients, medical_records) estratta dinamicamente tramite ispezione profonda dei pacchetti (DPI).

2.5 Orchestrazione e Isolamento dei Container

L'intera architettura è stata implementata e ingegnerizzata utilizzando **Docker** e **Docker Compose**, per garantire la massima portabilità e un isolamento rigoroso a livello di sistema operativo. Come mostrato nel `docker-compose.yaml`, l'infrastruttura è composta da diversi container, ognuno con un ruolo specifico e confinato in base al principio del minimo privilegio.

Le reti virtuali agiscono come veri e propri *VLAN tagging* all'interno dello stack Docker. Ad esempio, il container `zta-mongodb` è collegato esclusivamente alla rete `backend-net`, rendendolo inaccessibile a chiunque non possieda un'interfaccia sulla stessa rete (come `envoy`). Analogamente, il `zta-firewall` espone un'interfaccia su `frontend-net`, `backend-net` ed `external-net`, fungendo da gateway per instradare o dropare i pacchetti ancor prima che raggiungano il livello applicativo.

L'isolamento non è limitato solo alla rete, ma si estende ai namespace e ai privilegi: il container `zta-snort` (IDS) condivide lo stesso *network namespace* di `Envoy` (`network_mode: "service:envoy-pep"`), permettendogli di intercettare passivamente tutto il traffico che transita per il proxy, senza però avere la possibilità di interferire o bloccare direttamente la comunicazione.

2.6 Il server database di risorsa (MongoDB)

Il nucleo di memorizzazione persistente dei dati clinici, denominato *secure core*, è costituito dal database orientato ai documenti `mongodb`, ospitato nel container `zta-mongodb` e isolato interamente all'interno della rete privata `backend-net`.

Il database aziendale gestisce in primis la collezione `patients`, la quale memorizza le cartelle cliniche dei pazienti. Ciascun documento contiene dati anagrafici e clinici di base come `patient_id`, `name`, `age`, `ward` (reparto di ricovero), `blood_type` (gruppo sanguigno) e informazioni altamente confidenziali come il piano terapeutico. Nelle iterazioni iniziali del progetto, lo stato di fiducia dei dispositivi veniva memorizzato staticamente all'interno del database (tramite collezioni dedicate allo ZTA Policy Store); nell'architettura definitiva, tale approccio è stato superato demandando la valutazione della *trust* interamente ai modelli predittivi del SIEM (`splunk MLTK`). Questa scelta trasforma il database in un puro contenitore di dati operativi, rispettando pienamente il principio di separazione delle responsabilità (*Separation of Concerns*).

Dal punto di vista dell'accesso di rete, il container del database non espone alcuna porta verso l'esterno né è direttamente raggiungibile dai client. Qualsiasi interrogazione diretta deve transitare obbligatoriamente attraverso il modulo proxy envoy, che intercetta le chiamate sulla porta standard 27017. Sfruttando il filtro di rete specifico per MongoDB (`envoy.filters.network.mongo_proxy`), il proxy decodifica il protocollo di comunicazione (inclusi i moderni OP_MSG) ed estrae il payload delle query. Ciò consente ad OPA di eseguire l'enforcement tramite *Deep Packet Inspection* (L7 DPI), applicando politiche di blocco istantaneo su query distruttive (es. `dropDatabase` o cancellazioni massive), proteggendo l'integrità del dato ancor prima che la richiesta raggiunga il motore di storage. L'accesso alle informazioni cliniche sensibili viene invece filtrato a monte sui rispettivi endpoint delle Web API HTTP.

2.7 Le Web API e il portale client di simulazione

Il frontend dell'applicazione e la logica di business sono stati ingegnerizzati per simulare l'interazione quotidiana in un ambiente ospedaliero reale e validare l'efficacia del perimetro Zero Trust.

Il backend applicativo è implementato in Python tramite il framework FastAPI nel container `zta-web-api`. Oltre a fungere da ponte verso il database MongoDB per l'estrazione delle cartelle cliniche tramite gli endpoint `/api/patients` e `/api/patients/sensitive`, il servizio assume formalmente il ruolo di **Policy Information Point (PIP)** all'interno del framework ABAC. Ricevuta la query formattata da OPA (contenente le dimensioni contestuali *ZTA 6D*), il PIP arricchisce dinamicamente il *payload* iniettandovi le 6 variabili comportamentali dell'utente necessarie per l'analisi del rischio: `failed_logins` (tasso di fallimento nelle ultime 24h), `session_freq` (frequenza di sessione nell'ultima ora), `hour_of_day` e `is_night` (indicatori di contesto temporale), `sensitivity_level` (classificazione di criticità della risorsa target) e `days_inactive` (indice di inattività dell'account). La scelta ingegneristica di demandare il tracciamento di tale stato comportamentale all'API, anziché calcolarlo nativamente all'interno di Splunk tramite query di aggregazione sui log storici, risponde a precise esigenze di ottimizzazione architetturale. In un ambiente di produzione prototipale, infatti, l'interrogazione continua di indici di log per ogni singola transazione introdurrebbe una latenza incompatibile con i severi vincoli temporali della *Continuous Authorization*. Di conseguenza, l'API funge da strato di mediazione ad altissima velocità: gestisce le metriche comportamentali e le fornisce pre-elaborate all'endpoint di Splunk MLTK, il quale viene interrogato esclusivamente nella sua veste di motore di inferenza matematica per ottenere la predizione istantanea del rischio, restituita poi al motore decisionale.

Il portale web a cui accedono i medici è erogato in modo indipendente per ciascun client simulato (`client-alice`, `client-bob`, `client-charlie`) tramite container dedicati che eseguono un server web locale (`simulator.py`) e presentano interfacce in HTML/CSS moderno. I tre client simulano diverse posture di sicurezza. L'utente Alice rappresenta un medico operante da una workstation d'ufficio interna autorizzata, dotata di chip TPM e attestazione hardware valida sulla subnet `10.0.1.0/24`. Alice accede correttamente al portale, potendo consultare senza limitazioni sia i dati clinici standard che quelli sensibili. Il client di Bob descrive l'accesso tramite un dispositivo sprovvisto di attestazione crittografica TPM: OPA intercetta la mancanza del requisito hardware e blocca la sessione fin dal form di login (errore 403), applicando il

principio di *Short-Circuiting* per evitare chiamate ridondanti al SIEM. Infine, il client di Charlie simula un operatore in modalità *smart working* connesso da una rete esterna (192.168.100.0/24). Pur essendo fuori dal perimetro aziendale, Charlie dispone di un dispositivo sicuro con TPM; il sistema gli concede quindi l'accesso di base al portale, avendo un rischio inferiore alla soglia ristretta per gli utenti remoti (soglia ≤ 26).

Per convalidare sul campo le politiche impostate, il sistema implementa un approccio di *Defense in Depth* (Difesa in Profondità) per contrastare gli attacchi. Gli attacchi di livello applicativo (L7) tramite le Web API vengono intercettati in prima linea dal PEP: la barra di ricerca del portale invia input contenenti parole chiave distruttive (es. DROP o DELETE) che generano una richiesta HTTP anomala, la quale viene ispezionata e bloccata istantaneamente dal filtro Lua nativo di Envoy. Per il traffico diretto verso il database, è invece OPA a eseguire la *Deep Packet Inspection* (DPI) sui payload estratti dal proxy sulla porta 27017. Le regole di autorizzazione continua di OPA intervengono anche per bloccare utenti legittimi ma contestualmente a rischio (es. Charlie su rete esterna bloccato sulla rotta /api/patients/sensitive). Infine, i tentativi di bypass a livello L4 eseguiti tramite terminale client (es. comandi curl diretti) vengono scartati dal firewall nftables, mentre il sistema IDS Snort notifica l'evento anomalo a Splunk.

CAPITOLO 3

Implementazione pratica e configurazione

In questo capitolo viene analizzata l'implementazione pratica dell'architettura Zero Trust, dettagliando le configurazioni dei singoli container e i meccanismi di interazione tra i moduli di controllo e protezione del traffico.

3.1 La microsegmentazione e il deploy con docker-compose

Come già discusso, il deploy dell'infrastruttura è orchestrato tramite la tecnologia Docker, definendo quattro reti virtuali distinte e isolate nel file `docker-compose.yaml`. Questa separazione impedisce il routing diretto tra i client esterni o di frontend e il secure core di backend, forzando il transito del traffico attraverso il proxy Envoy. per chiarezza riprendiamo la configurazione delle reti isolate definita nel file mostrato nel Listato 2.

```
●●● docker-compose.yaml
1  networks:
2    frontend-net:
3      driver: bridge
4      ipam:
5        config:
6          - subnet: 10.0.1.0/24
7    backend-net:
8      driver: bridge
9    control-plane-net:
10     driver: bridge
11    external-net:
12     driver: bridge
13     ipam:
14       config:
15         - subnet: 192.168.100.0/24
```

Listing 2: Definizione delle quattro reti segregate nel file `docker-compose.yaml`.

3.2 La configurazione di Envoy Proxy come PEP

Il proxy envoy adempie al ruolo fondamentale di **Policy Enforcement Point (PEP)** all'interno dell'architettura progettata, operando come un reverse proxy trasparente e sicuro incaricato di intercettare, decifrare e ispezionare la totalità del traffico dati diretto verso il *secure core*. La configurazione, interamente definita nel file `envoy.yaml`, implementa una rigorosa segregazione dei flussi mediante la dichiarazione di due listener principali operanti su porte distinte, scongiurando qualsiasi possibilità di bypass dei controlli:

- **Listener database (mongodb_listener):** istanziato sulla porta standard 27017, gestisce il traffico TCP nativo verso il database MongoDB. Il blocco `transport_socket` impone l'obbligatorietà del protocollo mutuo TLS (*mTLS*) configurando il parametro `require_client_certificate: true`, costringendo ogni client a presentare un certificato x509 valido per l'attestazione hardware.
- **Listener HTTP (http_listener):** operativo sulla porta 8443, intercetta le chiamate in ingresso dirette alle Web API ospedaliere, decodificando il protocollo HTTP/1.1 o HTTP/2 ed esponendo i servizi web protetti da cifratura end-to-end.

Al fine di estrarre in tempo reale le metriche contestuali (*ZTA 6D*) necessarie al calcolo del rischio comportamentale e abilitare i meccanismi di *Deep Packet Inspection* (DPI), Envoy si avvale di una struttura sequenziale nota come **Filter Chain** (Catena dei Filtri).

Nel listener del database, il flusso informativo attraversa tre moduli specializzati. Inizialmente, il filtro di rete `tls_inspector` interviene durante l'handshake TLS iniziale analizzando il pacchetto di `Client Hello` prima della decifrazione, calcolando l'hash crittografico delle suite di cifratura per ricavare il fingerprinting *JA3*. Successivamente, il modulo applicativo `mongo_proxy` decodifica il protocollo binario BSON (inclusi i moderni opcode `OP_MSG`), estraendo dinamicamente i metadati associati al comando eseguito e alla collezione target. Infine, il filtro `ext_authz` sospende l'elaborazione del socket e inoltra in modo sincrono un payload gRPC (HTTP/2) al server di autorizzazione esterno OPA, includendo sia il certificato client in formato PEM (per la validazione del modulo TPM) sia i metadati estratti da MongoDB. Un estratto dettagliato di tale architettura di rete è documentato nel Listato 3.

```

1 static_resources:
2   listeners:
3     - name: mongodb_listener
4       address:
5         socket_address:
6           address: 0.0.0.0
7           port_value: 27017
8       listener_filters:
9         - name: envoy.filters.listener.tls_inspector
10          typed_config:
11            "@type": type.googleapis.com/envoy.extensions.filters.
12              listener.tls_inspector.v3.TlsInspector
13            enable_ja3_fingerprinting: true
14          filter_chains:
15            - filters:
16              - name: envoy.filters.network.mongo_proxy
17                typed_config:
18                  "@type": type.googleapis.com/envoy.extensions.filters.
19                    network.mongo_proxy.v3.MongoProxy
20                  stat_prefix: mongodb
21                  emit_dynamic_metadata: true
22              - name: envoy.filters.network.ext_authz
23                typed_config:
24                  "@type": type.googleapis.com/envoy.extensions.filters.
25                    network.ext_authz.v3.ExtAuthz
26                  stat_prefix: opa_authz
27                  transport_api_version: V3
28                  grpc_service:
29                    envoy_grpc:
30                      cluster_name: opa_cluster
31                  timeout: 45s
32                  include_peer_certificate: true

```

Listing 3: Estratto della configurazione del listener MongoDB in envoy.yaml.

Per quanto concerne il listener HTTP protetto sulla porta 8443, la catena dei filtri applicativi (gestita dall'HttpConnectionManager) implementa un approccio strategico di **Defense in Depth** (Difesa in Profondità). Prima che la richiesta sia trasmessa al backend o sottoposta alla valutazione del contesto da parte di OPA, intervengono due filtri locali in linea. Il modulo `header_mutation` inietta proattivamente l'hash JA3 estratto all'interno dell'header HTTP personalizzato `x-client-fingerprint`.

Immediatamente dopo, interviene un firewall L7 programmabile basato sul motore lua (`envoy.filters.http.lua`). Tale componente agisce come un filtro stateless ad altissima velocità, incaricato di validare la struttura formale della richiesta. Come evidenziato nel codice reale riportato nel Listato 4, lo script intercetta preventivamente i tentativi di attacco basati su metodi HTTP distruttivi (es. chiamate di tipo DELETE dirette agli endpoint dei pazienti), respingendoli immediatamente con un codice di errore 403 Forbidden. Questa stratificazione consente di sgravare il PDP (OPA) e i modelli di Machine Learning di Splunk dal computo di richieste palesemente malevole, ottimizzando la latenza complessiva del sistema. Le logiche di autorizzazione contestuale granulari

— come la limitazione d’accesso alla rotta sensibile per gli utenti in smart working — vengono invece delegate al successivo modulo `ext_authz`, preservando la separazione tra filtri strutturali e decisioni adattive.

envoy.yaml (Filtro HTTP Lua)

```

1      - name: envoy.filters.http.lua
2        typed_config:
3          "@type": type.googleapis.com/envoy.extensions.filters.
4            http.lua.v3.Lua
5          inline_code: |
6            function envoy_on_request(request_handle)
7              local path = request_handle:headers():get(":path")
8              local method = request_handle:headers():get(":
9                method")
10
11              -- L7 DPI Firewall Rules
12              if method == "DELETE" and string.match(path, "^/api
13                /patients") then
14                request_handle:respond({[":status"] = "403"}, '{
15                  "error": "L7 Firewall Block: Destructive operations (DROP) are
16                    forbidden"}')
17              end
18            end
19          end

```

Listing 4: Configurazione del filtro HTTP Lua in `envoy.yaml` per la prima linea di difesa L7.

3.3 Formalizzazione del Controllo degli Accessi: Modello ABAC e Matrice Dinamica

Il perimetro di sicurezza progettato supera il concetto statico di autorizzazione per implementare un modello dinamico riconducibile all’architettura **ABAC** (*Attribute-Based Access Control*). In accordo con la teoria del controllo degli accessi e il modello di Harrison, Ruzzo e Ullman (HRU), il dominio di protezione è definito da tre insiemi fondamentali:

- L’insieme dei **Soggetti** $S = \{s_1, \dots, s_n\}$, che rappresentano le entità computazionali richiedenti (es. gli operatori ospedalieri o i microservizi).
- L’insieme degli **Oggetti** $O = \{o_1, \dots, o_m\}$, che costituiscono le risorse da proteggere (es. endpoint HTTP, database, cartelle cliniche).
- L’insieme dei **Diritti** $R = \{r_1, \dots, r_k\}$, che definiscono le operazioni eseguibili (es. GET, POST, find, drop).

Nel modello HRU classico, il sistema dei permessi è rappresentato da una **Matrice di Access Control** bidimensionale A , dove ciascuna voce $A[s_i, o_j] \subseteq R$ indica staticamente i diritti che il soggetto s_i possiede sull’oggetto o_j . L’operazione r è consentita se e solo se $r \in A[s_i, o_j]$.

Nell’architettura Zero Trust sviluppata, tale approccio discrezionale e statico viene superato in favore del paradigma ABAC: le identità e il contesto di rete non fungono

più da indici statici per la matrice, ma vengono tradotti in un *insieme di attributi*. Come definito dalla teoria, nell'ABAC i permessi non sono semplici associazioni preconfigurate, bensì *espressioni booleane (condizioni) valutate dinamicamente sugli attributi* del richiedente.

Di conseguenza, la cella della matrice $A[s_i, o_j]$ cessa di essere una locazione di memoria fissa per trasformarsi in una funzione di autorizzazione calcolata in tempo reale dal Policy Decision Point (OPA), dipendente dallo stato corrente degli attributi:

$$A[s_i, o_j] = \{r \in R \mid D(s_i, o_j, r, \text{Attributi}) = \text{True}\}$$

A titolo esemplificativo, la Tabella 1 illustra un'istanza logica della matrice in un dato istante t , evidenziando come i diritti mutino in base al contesto.

Matrice di Protezione A	o_1 (Cartelle Standard)	o_{sens} (Dati Sensibili)	o_{db} (Database)
s_1 (Alice: Intranet, TPM)	{GET, POST}	{GET}	{find}
s_2 (Charlie: Esterno, TPM)	{GET}	$\emptyset$	$\emptyset$
s_3 (Attaccante: No TPM)	$\emptyset$	$\emptyset$	$\emptyset$

Tabella 1: Esempio di istanza dinamica della matrice HRU basata sul calcolo ABAC.

Le *Corporate Security Policy* sono state formalizzate in OPA traducendo le seguenti proposizioni logiche inderogabili sui flussi di informazione:

Policy 1: Requisito Hardware di Autenticazione (Short-Circuit)

Nessun diritto di accesso iniziale può essere concesso in assenza di una radice di fiducia hardware verificata. Sia $s_{tpm} \in \{0, 1\}$ l'attributo che certifica la presenza del chip TPM:

$$\forall o \in \text{Login}, \quad \text{se } s_{tpm} = 0 \implies A[s, o] = \emptyset$$

Policy 2: Autorizzazione Adattiva (Maximum e Low Trust)

L'assegnazione dei diritti dipende dal punteggio di rischio $s_{risk} \in [0, 100]$ calcolato algoritmicamente (Splunk MLTK) e dalla provenienza di rete s_{net} . Per un utente interno con TPM, la matrice abilita i diritti finché il rischio si mantiene entro l'intervallo di confidenza statistica $(\mu + 3\sigma)$.

$$\text{se } s_{net} = \text{Internal} \wedge s_{tpm} = 1 \wedge s_{risk} \leq 38 \implies r \in A[s, o]$$

Al contrario, per scenari di Smart Working ($s_{net} = \text{External}$), si impone il principio del guinzaglio corto $(\mu + 1.5\sigma)$:

$$\text{se } s_{net} = \text{External} \wedge s_{tpm} = 1 \wedge s_{risk} \leq 26 \implies r \in A[s, o]$$

Policy 3: Confidenzialità e Segregazione del Dato Sensibile (DPI Livello 1)

In accordo con le politiche di flusso delle informazioni (impedire letture non autorizzate verso domini non fidati), l'accesso alle risorse altamente sensibili (o_{sens}) è vincolato a requisiti stringenti. Se l'utente opera da rete esterna, la cella della matrice viene svuotata dinamicamente:

$$\text{se } o = o_{sens} \wedge (s_{net} \neq \text{Internal} \vee s_{tpm} = 0) \implies \text{GET} \notin A[s, o_{sens}]$$

Policy 4: Integrità e Protezione Attiva (DPI Livello 2)

Qualsiasi diritto classificato come distruttivo ($r \in \{\text{drop}, \text{deleteall}\}$) non può mai essere concesso all'interno del flusso applicativo, garantendo la disponibilità e l'integrità del dato clinico:

$$\forall s \in S, \forall o \in O, \quad \{\text{drop}, \text{deleteall}\} \notin A[s, o]$$

In sintesi, la matrice degli accessi A muta la propria topologia adattandosi ad ogni transazione. La cella che relaziona l'utente in smart working alla cartella clinica conterrà il diritto di lettura GET fintantoché il rischio comportamentale sarà accettabile, ma subirà la revoca istantanea (la cella diverrà l'insieme vuoto $\emptyset$) non appena si manifesterà un'anomalia nel SIEM o una disconnessione dalla rete sicura.

3.4 Il motore decisionale OPA (PDP) e l'integrazione Splunk MLTK

Il Policy Decision Point (PDP) riceve la richiesta di autorizzazione da Envoy ed esegue la valutazione delle policy definite in linguaggio Rego all'interno del file `rules.rego`. Il cuore del sistema è progettato per supportare la *Continuous Authorization* basata sul contesto, implementando quello che in letteratura è definito un **Contextual and Score-based Trust Algorithm**. Per fare ciò, OPA interroga sincronicamente l'API predittiva di Splunk MLTK per ottenere una valutazione istantanea della fiducia basata sulla storia recente e sul comportamento dell'utente.

La query predittiva, strutturata nel linguaggio SPL (*Search Processing Language*) di Splunk come mostrato nel Listato 5, viene interpolata dinamicamente da OPA inserendo le sei dimensioni correnti (ZTA 6D) estratte dai metadati della connessione.

●●● Splunk SPL Query (Interpolata dinamicamente da OPA)

```
1 | makeresults
2 | eval user="Alice", software="JA3_HASH", device="Workstation
   Ospedaliera Sicura (TPM Validato)", network="10.0.1.5", action="
   find", resource="patients"
3 | apply trust_model
4 | rename "predicted(rischio)" as rischio
5 | table rischio
```

Listing 5: Struttura della query Splunk MLTK generata dinamicamente da OPA.

Per ottimizzare le prestazioni architetturali e prevenire la saturazione del SIEM con traffico evidentemente malevolo, la chiamata al modello di Machine Learning è preceduta da una **clausola di guardia** (*Short-Circuiting*) che agisce come un **Criteria-based Trust Algorithm**. Se una richiesta non soddisfa i criteri infrastrutturali di base (ad esempio, un tentativo di login senza attestazione TPM), OPA interrompe la valutazione, assegna il rischio massimo di default e rifiuta la connessione senza interpellare Splunk.

Accanto all'analisi comportamentale, OPA implementa un motore di *Deep Packet Inspection* (L7 DPI) strutturato su due livelli di difesa per garantire la corretta imposizione degli obiettivi di **Confidenzialità** e **Integrità** sui flussi di informazione, come illustrato nel Listato 6. Il primo livello (Confidenzialità) interviene sul traffico HTTP per

segregare l'accesso ai dati sensibili dei pazienti (/api/patients/sensitive), bloccando preventivamente i flussi verso domini non fidati se l'utente proviene da una rete esterna o è sprovvisto di modulo TPM. Il secondo livello (Integrità) protegge il database decodificando i payload nativi BSON ed inibendo l'esecuzione di query distruttive dirette (come dropdatabase), prevenendo alterazioni di stato non autorizzate.

●●● rules.rego (DPI L7 e Protezione Sensibile)

```

1 # L7 DPI: Regole di blocco esplicite su traffico HTTP e MongoDB
2 default l7_dpi_block := false
3
4 # --- LIVELLO 1: Policy di Confidenzialita (Endpoint HTTP Sensibili)
5   ---
6 l7_dpi_block := true if {
7   is_http
8   resource == "/api/patients/sensitive"
9   not is_internal_network # Blocca reti esterne
10 } else := true if {
11   is_http
12   resource == "/api/patients/sensitive"
13   not is_tpm # Blocca assenza TPM
14 }
15
16 # --- LIVELLO 2: Policy di Integrita (Comandi nativi MongoDB) ---
17 else := true if {
18   is_mongo
19   contains(lower(db_query), "dropdatabase")
20 } else := true if {
21   is_mongo
22   contains(lower(db_query), "deleteall")
23 }

```

Listing 6: Regole di ispezione profonda L7 DPI per confidenzialità e integrità in rules.rego.

Il verdetto finale di accesso (allow) viene deliberato calcolando un livello di confidenza basato sullo stato dell'hardware, la localizzazione di rete e il punteggio dinamico di Splunk (splunk_risk_score), come dettagliato nel Listato 7. Le soglie di tolleranza deterministiche implementate (26 e 38) non sono arbitrarie, ma derivano rigorosamente dall'analisi statistica della distribuzione normale (*curva di Gauss*) dei log storici legittimi elaborati dall'algoritmo del SIEM.

Nello specifico, il valore di 38 rappresenta la soglia di tolleranza massima assegnata a un utente interno ideale dotato di attestazione hardware (*Alice*). Tale barriera viene calcolata impostando un intervallo di confidenza pari a tre deviazioni standard oltre la media del comportamento di baseline ($\mu + 3\sigma$), una metrica statistica che scherma il 99.7% delle fluttuazioni operative ordinarie riducendo a zero i falsi positivi, ma intercettando tempestivamente anomalie critiche dovute a sessioni compromesse.

Al contrario, la soglia ridotta a 26 risponde al principio Zero Trust del **guinzaglio corto** (*Low Trust Tolerance*), applicato agli scenari intrinsecamente esposti a maggior rischio perimetrale: gli utenti legittimi in smart working (*Charlie*) e i dispositivi legacy privi di TPM operanti tramite fallback su fingerprinting JA3. Poiché l'algoritmo di

generazione del dataset impone a chiunque operi da rete esterna una penalità infrastrutturale fissa di partenza pari a 18, la soglia di 26 limita il margine comportamentale consentito a sole 1.5 deviazioni standard oltre la media ($\mu + 1.5\sigma$). Questo garantisce che una minima deviazione dalla routine quotidiana provochi l'immediato superamento della barriera e il conseguente blocco della transazione da parte di OPA. Infine, per l'autenticazione iniziale (`is_auth_request`), viene meno qualsiasi approccio *Score-based* adattivo e si impone l'obbligo *Criteria-based* statico e vincolante del modulo TPM, neutralizzando gli attacchi brute-force prima ancora della valutazione del rischio.

●●● rules.rego (Regole di autorizzazione adattiva)

```

1 # La regola di autorizzazione adattiva (Risk-Based & JA3 Fallback)
2 risk_ok := true if {
3     is_internal_network
4     is_tpm
5     splunk_risk_score <= 38
6 } else := true if {
7     is_internal_network
8     not is_tpm
9     software != "Sconosciuto"
10    splunk_risk_score <= 26
11 } else := true if {
12     not is_internal_network
13     is_tpm
14     splunk_risk_score <= 26
15 } else := false
16
17 # Blocca preventivamente l'autenticazione senza TPM
18 is_auth_blocked := true if {
19     is_auth_request
20     software == "Sconosciuto"
21 } else := true if {
22     is_auth_request
23     not is_tpm # <--- BLOCCO TASSATIVO LOGIN SENZA TPM
24 } else := false

```

Listing 7: Regole di autorizzazione adattiva (Score-based) e short-circuiting (Criteria-based) in rules.rego.

3.5 Regole di network protection e tracciamento su nftables

Il componente firewall, basato sul framework `nftables` del kernel Linux, agisce come barriera L3/L4 di frontiera. Esso blinda l'architettura forzando l'instradamento obbligatorio verso il *Policy Enforcement Point* (Envoy) e gestendo una trappola attiva per rilevare e stroncare i tentativi di bypass della crittografia o del proxy.

All'avvio del container tramite lo script `start.sh`, il firewall esegue quattro task architetturali primari:

1. **Risoluzione dinamica:** interroga il resolver DNS interno di Docker per ottenere l'IP assegnato dinamicamente al container del proxy (`zta-envoy`).

2. **Segregazione e NAT (Network Address Translation):** reindirizza tutto il traffico TCP in ingresso verso l'IP di Envoy sulla porta sicura 8443 tramite una regola di *Destination NAT* (DNAT) operante nella catena di prerouting, applicando contestualmente il mascheramento SNAT per garantire la corretta commutazione dei pacchetti di ritorno.
3. **Sinkhole e Trappola di bypass (L4):** intercetta qualsiasi tentativo di connessione diretta (spesso sintomo di movimento laterale) alle porte di backend non autorizzate (come la porta 8000 delle Web API). Il traffico anomalo genera prima un log a livello kernel verso il demone `ulogd2` con il prefisso `[NFT-BLOCK]`, per poi essere scartato istantaneamente (drop).
4. **Automated Remediation (Denylist dinamica):** istanzia un set denominato `denylist` per il blocco assoluto (forward drop) degli indirizzi IP compromessi, la cui popolazione avviene in tempo reale.

La configurazione delle catene, dei set e delle regole di NAT applicate all'avvio del firewall è mostrata nel Listato 8.

●●● **start.sh (Configurazione nftables)**

```

1 # --- Inizializzazione Base Nftables ---
2 nft flush ruleset
3 nft add table ip filter
4 nft add set ip filter denylist { type ipv4_addr \; }
5 nft add chain ip filter forward { type filter hook forward priority 0
6     \; policy accept \; }
7
8 # Regole di DROP assoluto per gli IP bannati
9 nft add rule ip filter forward ip saddr @denylist counter drop
10
11 # --- Port Forwarding (DNAT) verso Envoy (PEP) ---
12 nft add table ip nat
13 nft add chain ip nat prerouting { type nat hook prerouting priority
14     -100 \; }
15 nft add chain ip nat postrouting { type nat hook postrouting priority
16     100 \; }
17 nft add rule ip nat prerouting tcp dport 8443 counter dnat to
18     $ENVOY_IP:8443
19 nft add rule ip nat postrouting ip daddr $ENVOY_IP tcp dport 8443
20     counter masquerade
21
22 # --- TRAPPOLA: Log e Drop del traffico L4 diretto al backend ---
23 nft add rule ip nat prerouting tcp dport 8000 log group 0 prefix "[
24     NFT-BLOCK] " counter
25 nft add rule ip nat prerouting tcp dport 8000 drop

```

Listing 8: Configurazione delle catene di filtering e NAT nello script di avvio del firewall.

Per abilitare l'inserimento dinamico in tale `denylist` senza la necessità di un intervento umano, il container del firewall ospita parallelamente un'API RESTful di management sviluppata in Python tramite il framework FastAPI. Questa interfaccia espone

l'endpoint POST `/ban`, il quale riceve un indirizzo IP e invoca il comando di sistema per iniettarlo direttamente nel set `denylist` del kernel, fornendo un meccanismo di reazione istantanea utilizzabile da sistemi esterni (come l'IDS Snort) in caso di rilevamento di traffico ostile.

Inoltre, il codice Python è strutturato nativamente per operare in sinergia con la pipeline di telemetria dell'architettura. Come definito nella libreria di logging, ogni richiesta di ban o intercettazione genera stringhe di log in formato chiave-valore (ad esempio `event="nftables_drop" status="added" client_ip="192.168.100.X"`). Questo formato agevola il parsing asincrono da parte del raccoglitore Fluent Bit, garantendo un'ingestion rapida e una visualizzazione strutturata all'interno delle dashboard del SIEM Splunk.

3.6 Autenticazione crittografica: mTLS, TPM e fingerprinting JA3

L'architettura implementa un meccanismo di verifica dell'identità a tre livelli complementari, che operano in modo congiunto durante la fase di handshake TLS per costruire le dimensioni u (utente), d (dispositivo) e s (software) della tupla contestuale Zero Trust.

3.6.1 Mutua autenticazione TLS (mTLS)

Ogni listener di Envoy impone la presentazione obbligatoria di un certificato client X.509 firmato dalla CA interna dell'ospedale (ZTA Hospital Trust Root CA). La direttiva `require_client_certificate: true` nella configurazione TLS di Envoy garantisce che nessuna connessione priva di certificato valido possa superare la fase di negoziazione. Il campo Common Name (CN) del certificato identifica univocamente l'operatore (es. `employee-alice`, `employee-bob`) e viene estratto da Envoy per essere propagato al motore OPA attraverso l'attributo gRPC `source.principal` della richiesta `ExtAuthz`.

3.6.2 Attestazione hardware TPM

Per discriminare i dispositivi aziendali sicuri dai terminali personali o compromessi, il sistema utilizza un'estensione X.509 personalizzata con Object Identifier (OID) `1.3.6.1.4.1.9999.1`. Ai fini della presente implementazione prototipale, lo script `generate_identities.py` simula il processo di *hardware attestation* arricchendo dinamicamente i certificati destinati alle workstation sicure (come quelle di Alice e Charlie) con tale estensione. Il frammento di generazione è mostrato nel Listato 9.

 generate_identities.py

```

1 TPM_OID = ObjectIdentifier("1.3.6.1.4.1.9999.1")
2
3 def generate_leaf(base_dir, name, cn, ca_cert, ca_key, has_tpm=False)
4     :
5     key = rsa.generate_private_key(public_exponent=65537, key_size
6     =2048)
7     subject = x509.Name([x509.NameAttribute(NameOID.COMMON_NAME, cn)
8     ])
9     builder = x509.CertificateBuilder()
10    # ... (configurazione base del certificato)
11    if has_tpm:
12        builder = builder.add_extension(
13            x509.UnrecognizedExtension(
14                TPM_OID,
15                b"TPM-VERIFIED-HARDWARE-ROOT-OF-TRUST-PCR0-OK"
16            ), critical=False
17        )
18    cert = builder.sign(ca_key, hashes.SHA256())

```

Listing 9: Generazione dei certificati client con estensione TPM simulata.

Nel mondo reale in un ambiente di produzione aziendale, tale estensione verrebbe apposta dalla CA solo dopo aver crittograficamente validato una *Certificate Signing Request* (CSR) firmata direttamente dalla Endorsement Key (EK) di un chip TPM fisico. Questo approccio ingegneristico garantisce la proprietà di inesportabilità della chiave privata del client: essendo generata in hardware, la chiave non può essere esfiltrata e il certificato utente risulta indissolubilmente legato alla macchina autorizzata. A livello di policy, OPA valida questa garanzia di postura in tempo reale decodificando il certificato PEM e verificando la presenza dell'OID specifico nell'array delle estensioni (Listato 10).

 rules.rego

```

1 # C. DISPOSITIVO E TPM
2 default is_tpm := false
3 is_tpm := true if {
4     cert_raw := input.attributes.source.certificate
5     cert_pem := urlquery.decode(cert_raw)
6     certs := crypto.x509.parse_certificates(cert_pem)
7     count(certs) > 0
8     some ext in certs[0].Extensions
9     ext.Id == [1, 3, 6, 1, 4, 1, 9999, 1]
10 }
11 default device := "Dispositivo non censito (No TPM)"
12 device := "Workstation Ospedaliera Sicura (TPM Validato)" if { is_tpm
13 }

```

Listing 10: Validazione dell'estensione TPM nel certificato client da parte di OPA.

3.6.3 Fingerprinting software JA3

Il terzo livello di identificazione opera a livello di trasporto, in maniera del tutto passiva rispetto al client. Durante l'handshake TLS, il filtro `tls_inspector` di Envoy calcola l'hash **JA3**, un'impronta digitale deterministica basata sulla combinazione dei parametri proposti nel pacchetto di `ClientHello` (versione TLS, cipher suite offerte, estensioni, curve ellittiche e formati dei punti). Poiché ogni implementazione software genera un `ClientHello` caratteristico, l'hash JA3 consente di distinguere un browser autorizzato (es. Firefox o Chrome) da uno strumento per l'enumerazione o l'attacco (es. `curl` o `nmap`), neutralizzando le tecniche di spoofing dell'header `User-Agent`. L'impronta calcolata viene propagata ad OPA tramite l'header `x-client-fingerprint` (su protocollo HTTP) o tramite i metadati dinamici di Envoy (su protocollo MongoDB).

Tuttavia, l'impiego del solo fingerprinting software presenta un limite strutturale intrinseco qualora applicato ad operatori umani: due dispositivi distinti che utilizzino il medesimo browser ed eseguano la stessa versione di sistema operativo produrranno un hash JA3 perfettamente identico, risultando crittograficamente indistinguibili.

Proprio alla luce di questa vulnerabilità, il motore decisionale Zero Trust classifica il JA3 come un indicatore di rischio debole per gli accessi interattivi, ma lo sfrutta strategicamente come **meccanismo di fallback per il traffico Machine-to-Machine (M2M) e per i dispositivi Kiosk/Legacy**. In scenari aziendali reali, molti sensori IoT o microservizi automatizzati (client *headless*) non dispongono fisicamente di un chip TPM, ma eseguono script deterministici (es. librerie Python `requests` o `pymongo`) che generano un fingerprint JA3 estremamente specifico e costante nel tempo.

Il framework gestisce queste eccezioni infrastrutturali tramite un'apposita policy di *JA3 Fallback*. Quando il PEP intercetta una connessione sprovvista di attestazione TPM ma recante un hash JA3 censito e associato a un'identità M2M nota, il PDP (OPA) non blocca istantaneamente la richiesta. Al contrario, degrada il livello di fiducia applicando il principio del "guinzaglio corto": la soglia di tolleranza del *Risk Score* viene abbassata al valore minimo (*soglia* ≤ 26) e vengono inibiti a priori i diritti di accesso verso endpoint sensibili o operatività distruttive. Questo approccio garantisce la continuità operativa per i servizi automatizzati e legacy, confinandoli simultaneamente in un perimetro di *Least Privilege* per mitigarne il rischio di compromissione.

3.7 Rilevamento delle intrusioni di rete: Snort NIDS

A completamento delle difese applicative gestite dal proxy e dal motore OPA, l'architettura integra snort come **Network Intrusion Detection System (NIDS)** passivo, posizionato strategicamente in modo da intercettare tutto il traffico perimetrale.

La tecnica di deploy adottata è quella della condivisione del *network namespace*: nel file `docker-compose.yaml`, il container `zta-snort` è configurato con la direttiva `network_mode: "service:envoy-pep"`. Questo approccio di ingegneria di rete consente all'IDS di agganciarsi direttamente all'interfaccia virtuale di Envoy, osservando passivamente tutti i pacchetti in ingresso e in uscita senza possedere un proprio indirizzo IP e senza introdurre latenza, agendo come una vera e propria sonda invisibile (TAP di rete).

All'avvio, Snort viene lanciato in ascolto sull'interfaccia principale del namespace condiviso, riversando l'output dei log in formato compatibile nella directory `/var/log/snort`, montata come volume Docker per l'ingestion asincrona da parte di Fluent-Bit. Le regole personalizzate, definite nel file `local.rules`, superano il tradizio-

nale rilevamento statico per intercettare anomalie volumetriche, tentativi di elusione crittografica e comportamenti operativi sospetti. L'intero set di firme è riportato nel Listato 11.

```

1 # 1. Rileva una scansione Nmap (SYN Scan) verso Envoy o DB
2 alert tcp any any -> any (msg:"ZTA ALERT: Rilevata Scansione di
   Rete (Possibile Nmap)"; \
3   flags:S; threshold:type both, track by_src, count 20, seconds 10;
   sid:1000001; rev:1;)
4
5 # 2. AUDIT: Traccia l'accesso tramite client nativo MongoDB
6 alert tcp any any -> any 27017 (msg:"ZTA AUDIT: Connessione Client
   nativa a MongoDB \
7   rilevata (Accesso non-Web)"; sid:1000002; rev:2;)
8
9 # 3. Rileva attacco volumetrico (Possibile DDoS / SYN Flood verso
   Envoy)
10 alert tcp any any -> any 8443 (msg:"ZTA ALERT: Anomalia volumetrica (
   Possibile DoS \
11   verso il Proxy)"; flags:S; threshold:type both, track by_src, count
   100, seconds 5; \
12   sid:1000003; rev:1;)
13
14 # 4. Rileva un tentativo di connessione in chiaro (Downgrade Attack
   HTTP)
15 alert tcp any any -> any 8443 (msg:"ZTA ALERT: Tentativo di traffico
   in chiaro su porta \
16   protetta (Downgrade/Misconfig)"; flow:established,to_server;
   content:"GET "; depth:4; \
17   sid:1000004; rev:1;)
18
19 # 5. Rileva tentativi ripetuti di connessione (Possibile Brute-Force
   del mTLS)
20 alert tcp any any -> any 8443 (msg:"ZTA ALERT: Eccessivi tentativi di
   connessione \
21   (Possibile assenza di Certificato Client)"; flags:R; threshold:type
   both, track by_src, \
22   count 15, seconds 10; sid:1000005; rev:1;)

```

Listing 11: Regole di rilevamento avanzate (Volumetriche, Protocolлари e Crittografiche) per Snort.

Il file di configurazione implementa quattro vettori logici di analisi principali:

- **Attacchi volumetrici e Reconnaissance (Regole 1 e 3):** Sfruttano il costrutto threshold per identificare anomalie basate sulla frequenza dei pacchetti di sincronizzazione (SYN). La Regola 1 identifica i pattern tipici degli scanner di rete come Nmap, mentre la Regola 3 monitora burst aggressivi diretti esclusivamente al listener web (8443), mitigando visivamente tentativi di *Denial of Service* (SYN Flood) mirati ad esaurire le risorse del proxy.

- **Anomalie Protocolлари e Downgrade (Regola 4):** Esegue una *Deep Packet Inspection* (DPI) sui primi byte del payload TCP alla ricerca di metodi in chiaro (come la stringa "GET "). Rilevare traffico non decifrato su una porta esplicitamente vincolata al TLS indica un'errata configurazione del client o un malintenzionato che tenta un *Downgrade Attack* forzando l'uso del protocollo HTTP standard.
- **Auditing Operativo (Regola 2):** Il sistema rileva attività legittime ma deviatrici rispetto alla baseline. Monitorando gli accessi sulla porta 27017, Snort genera eventi di Audit quando operatori autorizzati (es. personale di Data Science) si collegano direttamente al database tramite client nativi, aggirando le Web API ma passando regolarmente per i controlli mTLS e OPA del proxy.

3.8 Aggregazione centralizzata dei log: Fluent-Bit

La raccolta e la centralizzazione dei log di telemetria provenienti dai diversi componenti dell'infrastruttura è affidata a *fluent-bit*, un agente di log forwarding leggero e ad alte prestazioni. Fluent-Bit è stato preferito al classico *syslog* per la sua capacità nativa di operare in ambienti containerizzati, supportando il parsing strutturato di log JSON e l'invio diretto verso Splunk tramite il protocollo **HTTP Event Collector** (HEC).

La configurazione, definita nel file *fluent-bit.conf*, implementa una pipeline di raccolta multi-sorgente con cinque canali di input e un'unica destinazione di output, come schematizzato nella Tabella 2.

Tabella 2: Pipeline di raccolta log di Fluent-Bit: sorgenti e tag associati.

Sorgente	Tag	Percorso Volume	Host Iniettato
Snort IDS	snort.alerts	/var/log/snort/alert_json.txt	zta-snort
MongoDB	mongo.audit	/var/log/mongodb/mongod.log	zta-mongodb
Envoy PEP	envoy.access	/var/log/envoy/access.log	zta-envoy
OPA PDP	opa.decisions	/var/log/opa/decision.log	zta-opa
NFTables Firewall	nftables.kernel	/var/log/nftables/firewall.log	zta-firewall

L'architettura sfrutta i **volumi Docker condivisi** come meccanismo di trasporto: ogni container scrive i propri log in un volume dedicato (es. *envoy_logs*, *snort_logs*) e Fluent-Bit li legge in modalità *tail* dal percorso montato. Per i log strutturati di OPA, Fluent-Bit applica una catena di filtri dedicata: un primo filtro *grep* seleziona solo le righe contenenti il marcatore [OPA-PDP], un parser con espressione regolare estrae il payload JSON dalla stringa di log, e un secondo parser deserializza il JSON strutturato contenente la decisione, il risk score e le sei dimensioni ZTA.

Tutti i log vengono infine inoltrati verso l'unica destinazione: il **Splunk SIEM** tramite il modulo di output *splunk* configurato con il token HEC e la connessione TLS verso la porta 8088 del container *splunk-siem*.

3.9 Il modello predittivo di Machine Learning: Splunk MLTK e Gradient Boosting

Il calcolo del punteggio di rischio dinamico rappresenta il motore decisionale della valutazione adattiva continua dell'architettura Zero Trust. L'intelligenza predittiva è implementata all'interno del SIEM tramite lo **Splunk Machine Learning Toolkit** (ML-

TK), segnando il passaggio da un approccio di autorizzazione statico a un'analisi comportamentale in tempo reale.

3.9.1 Dall'approccio statico alla valutazione comportamentale continua

L'architettura Zero Trust di base si fonda su un'analisi del contesto di autenticazione composta da 6 dimensioni identitarie (*user, software, device, network, action, resource*). Per abilitare la *Continuous Authorization*, il modello integra ulteriori 6 feature comportamentali, iniettate dinamicamente dal proxy Web API. Tali feature trasformano la fotografia istantanea dell'accesso in un monitoraggio continuo della sessione:

- `failed_logins`: tentativi di accesso falliti nelle ultime 24 ore.
- `hour_of_day` e `is_night`: fascia oraria e flag per accessi in orario notturno (22:00–06:00).
- `session_freq`: frequenza delle richieste nell'ultima ora (indicatore anti-bot).
- `sensitivity_level`: grado di criticità della risorsa richiesta (es. dati pazienti vs configurazioni).
- `days_inactive`: giorni trascorsi dall'ultimo login riuscito.

3.9.2 Generazione del dataset e modello matematico

Il modello predittivo viene addestrato ed esaminato su un dataset sintetico composto da 50.000 record, strutturato per mappare accuratamente il comportamento di dipendenti legittimi (es. profili aziendali *alice, bob, charlie*) e pattern di attacco realistici di diversa intensità (*hacker_x, script_kiddie, insider_threat, apt_agent, intruder_7*).

Il valore target predittivo è rappresentato dal punteggio di rischio (espresso su una scala continua 0 – 100). Nel dataset di addestramento, tale valore viene calcolato tramite una funzione deterministica non lineare a tre livelli architetturali, che isola le debolezze strutturali dalle anomalie comportamentali, ponderando queste ultime in base alla criticità dell'asset invocato:

$$\text{Rischio} = \text{Rischio}_{\text{Infra}} + (\text{Rischio}_{\text{Comp}} \times M_{\text{Sens}}) + \epsilon \quad (2)$$

Il modello così concepito riflette severità architetturali precise, scorporando il calcolo nelle seguenti macro-componenti analitiche:

1. **Rischio Infrastrutturale ($\text{Rischio}_{\text{Infra}}$):** Valuta lo stato di conformità e sicurezza del dispositivo e del canale di comunicazione prima dell'analisi delle azioni. È definito come:

$$\text{Rischio}_{\text{Infra}} = \Delta_{\text{HW}} + \Delta_{\text{Rete}} + \Delta_{\text{Combo}} \quad (3)$$

- $\Delta_{\text{HW}} = +30$: Penalità applicata qualora il dispositivo sia sprovvisto di attestazione hardware tramite chip TPM (`missing_tpm` o `unknown`). Rappresenta il fattore di rischio strutturale più grave.
- $\Delta_{\text{Rete}} = +18$: Penalità per connessioni originata da reti esterne al perimetro ospedaliero (es. contesti di Smart Working, VPN domestiche o Wi-Fi pubblici).

- $\Delta_{\text{Combo}} = +7$: Fattore di correlazione critica cumulativo applicato esclusivamente qualora si verifichino simultaneamente l'assenza di TPM e la provenienza da rete esterna ($\Delta_{\text{HW}} > 0 \wedge \Delta_{\text{Rete}} > 0$), configurando uno scenario ad alto potenziale di compromissione.

2. **Rischio Comportamentale ($\text{Rischio}_{\text{Comp}}$):** Aggrega le anomalie riscontrate durante la sessione operativa attraverso la somma algebrica di sei indicatori di compromissione (IoC) e deviazioni dalla *baseline* utente:

$$\text{Rischio}_{\text{Comp}} = B_{\text{Logins}} + B_{\text{Night}} + B_{\text{Freq}} + B_{\text{Inact}} + B_{\text{Action}} + B_{\text{Soft}} \quad (4)$$

- *Autenticazioni Fallite (B_{Logins}):* Modellate tramite una funzione non lineare progressiva, logaritmica in base 2, vincolata superiormente a un tetto massimo di +25 punti per evitare la saturazione del rischio su singoli eventi isolati:

$$B_{\text{Logins}} = \min(\text{failed_logins} \times 3.5 + \log_2(\text{failed_logins} + 1) \times 2.5, 25) \quad (5)$$

- *Anomalia Oraria (B_{Night}):* Introduzione di un malus fisso di +5 punti qualora la richiesta avvenga in ambiente notturno, ovvero nell'intervallo temporale [22 : 00, 06 : 00).
- *Frequenza di Sessione (B_{Freq}):* Funzione a scaglioni discreti che penalizza i comportamenti automatici tipici di *botnet* o script di *scraping*:

$$B_{\text{Freq}} = \begin{cases} +7 & \text{se session_freq} > 30 \\ +4 & \text{se } 20 < \text{session_freq} \leq 30 \\ +2 & \text{se } 10 < \text{session_freq} \leq 20 \\ 0 & \text{altrimenti} \end{cases} \quad (6)$$

- *Inattività Prolungata (B_{Inact}):* Penalizzazione progressiva legata alla riattivazione di account dormienti, calcolata come +2 punti per ogni blocco di 15 giorni di inattività, fino a un massimo di +8:

$$B_{\text{Inact}} = \min(\lfloor \text{days_inactive} / 15 \rfloor \times 2, 8) \quad (7)$$

- *Severità dell'Operazione (B_{Action}):* Malus fisso di +8 punti qualora l'azione intercettata dal PEP preveda l'invocazione di comandi distruttivi o di alterazione strutturale sul database (delete, drop).
- *Software Sospetto (B_{Soft}):* Incremento di +4 punti nel caso in cui l'analisi dell'handshake o dell'User-Agent riveli strumenti di scansione o librerie non standard (nmap, metasploit, curl, script automatizzati in Python).

3. **Moltiplicatore di Sensibilità della Risorsa (M_{Sens}):** In linea con i paradigmi Zero Trust avanzati, l'impatto del rischio comportamentale viene linearmente amplificato in funzione della criticità della risorsa richiesta, lasciando inalterato l'indice se il comportamento è perfettamente allineato alla baseline ($\text{Rischio}_{\text{Comp}} \sim 0$). La funzione di amplificazione segue la legge:

$$M_{\text{Sens}} = 1.0 + (\text{sensitivity_level} - 1) \times 0.175 \quad (8)$$

I livelli di sensibilità sono strettamente mappati sul dominio applicativo ospedaliero:

- *Livello 1* ($M_{\text{Sens}} = 1.00$): Risorse a bassa criticità (es. collezione utenti).
- *Livello 2* ($M_{\text{Sens}} = 1.175$): Risorse a criticità moderata (es. collezioni pazienti, system_logs).
- *Livello 3* ($M_{\text{Sens}} = 1.35$): Risorse ad alto rischio e dati ultrasensibili (es. cartelle_cliniche, config_db).

Infine, il termine $\epsilon \sim \mathcal{N}(0, 2)$ introduce un rumore statistico gaussiano con media nulla e deviazione standard pari a 2. L'inserimento di tale fluttuazione stocastica, unito a un operatore di *clamping* rigido nell'intervallo $[0, 100]$, è di fondamentale importanza per l'addestramento del regressore (GradientBoostingRegressor) su Splunk MLTK: previene fenomeni di *overfitting* derivanti da un dataset puramente geometrico e costringe l'algoritmo a generalizzare le relazioni non lineari latenti tra le sei dimensioni del contesto ZTA.

3.9.3 Selezione dell'algoritmo: Gradient Boosting vs Random Forest

La scelta dell'algoritmo **Gradient Boosting Regressor** in contrapposizione a tecniche parallele (come la *Random Forest*) è motivata dall'intrinseca natura composita del rischio informatico.

Nel paradigma *Random Forest* (basato sul *bagging*), vengono creati molteplici alberi decisionali in parallelo e la predizione finale è la media aritmetica dei risultati. Sebbene efficace per ridurre la varianza, questa mediazione tende a "smussare" le anomalie, un difetto critico nel dominio della sicurezza dove un picco anomalo non deve essere attenuato. Al contrario, il Gradient Boosting opera secondo il principio dell'**apprendimento additivo sequenziale**. L'algoritmo costruisce un albero alla volta; ogni nuovo *weak learner* viene addestrato specificamente per approssimare i residui (gli errori di stima) commessi dall'albero precedente, minimizzando direttamente la *Loss Function* calcolando il gradiente dell'errore.

Questo approccio risulta superiore nel catturare le relazioni non lineari tra le 12 feature ZTA: un utente che fallisce 5 login di giorno dall'ufficio possiede un profilo di rischio moderato, ma lo stesso utente che fallisce 5 login alle 3 di notte da un IP esterno rappresenta una minaccia critica. Focalizzandosi sulla correzione degli errori sui casi limite, il Gradient Boosting penalizza severamente queste combinazioni atipiche.

3.9.4 Addestramento e Regressione in tempo reale

L'addestramento avviene direttamente nell'interfaccia di Splunk tramite il costruito SPL mostrato nel Listato 12. Una volta generato, il modello viene configurato con permessi *Globali* per consentirne l'interrogazione esterna tramite API REST.

Splunk SPL | Addestramento del Trust Model

```
1 | inputlookup simulated_traffic.csv
2 | fit GradientBoostingRegressor "rischio"
3   from user, software, device, network, action, resource,
4   failed_logins, hour_of_day, is_night,
5   session_freq, sensitivity_level, days_inactive
6   into trust_model
```

Listing 12: Query SPL per l'addestramento del modello Gradient Boosting su Splunk MLTK.

Il vantaggio operativo di questo approccio risiede nell'utilizzo della **Regressione** anziché della Classificazione. Il modello non restituisce un giudizio binario (sicuro/pericolo), ma un punteggio numerico continuo (es. 18.4, 42.7). Durante il funzionamento, OPA ingerisce dinamicamente questo valore per attuare politiche di autorizzazione adattive, applicando le soglie di tolleranza predefinite (es. blocco per rischio > 38 su rete interna o > 26 su rete esterna), rendendo il perimetro di sicurezza fluido e proporzionale alla postura reale dell'utente.

CAPITOLO 4

Simulazione degli scenari e validazione

Per verificare la robustezza dell'infrastruttura Zero Trust progettata, sono stati simulati quattordici scenari operativi, organizzati per livello di difesa (dalla crittografia di trasporto fino all'analisi comportamentale ML), ognuno mirato a testare specifiche componenti difensive e logiche del sistema. Tutti gli utenti ed operatori si interfacciano inizialmente con lo stesso portale web centralizzato, protetto da autenticazione forte e controlli contestuali (mostrato in Figura 3).

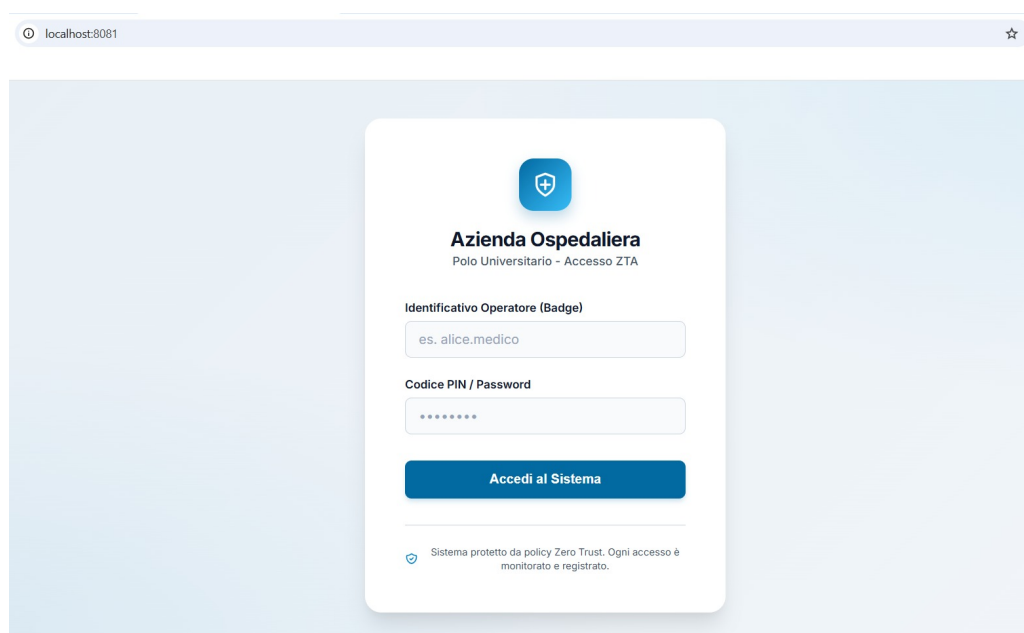


Figura 3: Schermata del portale di login protetto.

4.1 Scenario 1: Tentativo di connessione senza certificato client

Il primo scenario ha l'obiettivo di validare il livello più esterno della difesa (Layer 1), testando la robustezza del Policy Enforcement Point (Envoy) rispetto a tentativi di connessione privi di un'identità crittografica valida. In un'architettura Zero Trust, l'assunto di base è che la rete sia intrinsecamente ostile; pertanto, il sistema deve esigere la *Mutual TLS* (mTLS) prima di autorizzare o instradare qualsiasi traffico applicativo.

In questo test, viene simulata una richiesta tramite lo strumento *curl* da parte di un client che tenta di connettersi all'endpoint delle API senza fornire il proprio certificato client:

curl_no_cert.sh

```

1 $ docker exec zta-client-charlie curl -k https://zta-firewall:8443/
  api/patients
2 % Total    % Received % Xferd  Average Speed   Time    Time
  Time      Current                      Dload  Upload   Total   Spent
3
4 0          0    0     0    0    0    0    0    0  --:--:--  --:--:--
5 curl: (56) OpenSSL SSL_read: OpenSSL/3.5.6: error:0A00045C:SSL
  routines::tlsv13 alert certificate required, errno 0

```

Listing 13: Blocco preventivo a livello di trasporto per assenza di certificato client.

Come si evince dall'output (Listato 13), il proxy Envoy rileva immediatamente l'assenza del certificato durante la fase di *handshake* TLS. La connessione viene abortita a livello di trasporto con un alert fatale (*tlsv13 alert certificate required*), impedendo l'instaurazione del canale sicuro.

È fondamentale sottolineare l'efficienza architetturale di questo meccanismo: la richiesta HTTP (GET /api/patients) non viene mai elaborata. Il blocco avviene esclusivamente a livello crittografico al perimetro, garantendo che né il Policy Decision Point (OPA), né il motore di inferenza Machine Learning (Splunk), né tantomeno i servizi di backend vengano interpellati. Questo approccio *fail-closed* previene attacchi di esplorazione o *Denial of Service* (DoS) a livello applicativo, annullando totalmente l'*overhead* computazionale per la gestione di traffico malevolo palese.

4.2 Scenario 2: Tentativo di accesso con certificato non firmato dalla CA

Per verificare ulteriormente il controllo sulla catena di fiducia (*Trust Chain*), un secondo test simula un attaccante che tenta di presentare un certificato crittograficamente valido nella forma, ma generato autonomamente (*self-signed*) e non firmato dalla *Root CA* ospedaliera:

curl_fake_cert.sh

```

1 $ docker exec zta-client-charlie sh -c "openssl req -x509 -newkey rsa
  :2048 \
2 -keyout /tmp/fake.key -out /tmp/fake.crt -days 1 -nodes -subj '/CN=
  intruder' 2>/dev/null \
3 && curl -k --cert /tmp/fake.crt --key /tmp/fake.key https://zta-
  firewall:8443/api/patients"
4 curl: (56) OpenSSL SSL_read: OpenSSL/3.5.6: error:0A000418:SSL
  routines::tlsv1 alert unknown ca, errno 0

```

Listing 14: Rilevamento e blocco di un certificato estraneo alla Trust Chain ospedaliera.

In questo scenario, il proxy Envoy esegue una validazione rigorosa della *Certificate Authority* emittente. Non riscontrando corrispondenza con la *Root CA* interna configurata, il sistema interrompe immediatamente la connessione con l'errore `tlsv1 alert unknown ca`. Anche in questo caso, come nel precedente, il blocco avviene a livello di *handshake* TLS, garantendo che nessuna risorsa interna (OPA o backend) venga esposta all'attaccante.

4.3 Scenario 3: Tentativo di bypass L4 e rilevamento attivo

Il terzo scenario simula un tentativo di attacco in cui un utente esterno (o un dispositivo compromesso nella rete esterna, come Charlie) tenta di aggirare completamente il proxy Envoy (PEP) per connettersi direttamente al database o alle API di backend.

L'attaccante esegue una richiesta diretta tramite lo strumento `curl` dal proprio terminale verso la porta delle API:

```
curl_bypass.sh

1 $ docker exec -it zta-client-charlie curl -v --connect-timeout 3 http
   ://zta-firewall:8000/api/patients
2 * Host zta-firewall:8000 was resolved.
3 * Trying 192.168.100.4:8000...
4 * Connection timed out after 3002 milliseconds
5 * closing connection #0
6 curl: (28) Connection timed out after 3002 milliseconds
```

Listing 15: Simulazione di attacco bypass diretto tramite `curl` dal terminale di Charlie.

Come previsto, la connessione va in timeout poiché entra in azione il firewall. `nftables` rileva la connessione diretta non autorizzata e applica la regola della trappola attiva, eseguendo il DROP silente dei pacchetti. Tale evento viene registrato nei log del firewall (mostrati nel Listato 16), evidenziando l'etichetta `[NFT-BLOCK]` e tutti i parametri del pacchetto TCP bloccato sulla porta di destinazione 8000 (DPT) proveniente dal client (`SRC=10.0.1.4`).

Si osservi che, in questo scenario di bypass di rete a livello di trasporto, lo score di rischio di Splunk non viene calcolato né registrato all'interno dei log di decisione di OPA. La connessione viene infatti intercettata e interrotta a livello di rete e trasporto (Livelli 3 e 4 del modello ISO/OSI) ad opera del firewall di rete `nftables`. Poiché i pacchetti vengono scartati (DROP) in kernel space prima di poter raggiungere il proxy Envoy (PEP) e attivare la catena di autorizzazione applicativa di OPA, non si innesca alcuna chiamata al proxy delle Web API ed al modello di Machine Learning. Di conseguenza, l'unica traccia dell'evento ostile in Splunk sarà limitata alle telemetrie di blocco del firewall (`[NFT-BLOCK]`) ed agli allarmi NIDS generati da Snort.

nftables.log

```
1 Jun  2 17:03:04 f020514ab776 [NFT-BLOCK] IN=eth2 OUT= MAC=e6:27:66:
d6:b7:c9:06:c9:5e:11:ef:c1:08:00 SRC=10.0.1.4 DST=10.0.1.5 LEN=60
TOS=00 PREC=0x00 TTL=64 ID=15717 DF PROTO=TCP SPT=58384 DPT=8000
SEQ=1187313041 ACK=0 WINDOW=64240 SYN URGP=0 MARK=0
```

Listing 16: Dettaglio del log di blocco generato da nftables per il tentativo di bypass.

I log del firewall, insieme alle telemetrie di sistema, vengono raccolti e centralizzati in tempo reale da Fluent-Bit ed inoltrati al SIEM Splunk. La Figura 4 mostra la schermata del pannello di ricerca di Splunk Search & Reporting, in cui si può riscontrare l'avvenuta indicizzazione dell'evento di blocco [NFT-BLOCK] generato dall'host zta-firewall.

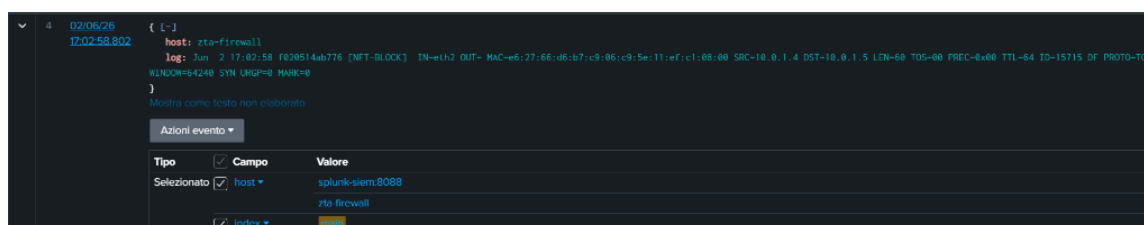


Figura 4: Schermata del pannello di ricerca di Splunk che mostra il log di blocco [NFT-BLOCK] indicizzato.

Contemporaneamente, la sonda NIDS snort intercetta la firma del pacchetto di attacco ed esegue l'ispezione DPI, generando un allarme di intrusione. I log di alert vengono centralizzati da Fluent-Bit ed inoltrati a Splunk SIEM, il quale correla l'evento elevando il livello di rischio associato all'indirizzo IP sorgente.

In uno scenario operativo completo, questa metrica di rischio innescherebbe un meccanismo di automazione (tramite API o webhook) verso il *Policy Decision Point* (OPA), aggiornando dinamicamente la *denylist* per isolare la minaccia a livello di *firewall*. Pur essendo l'infrastruttura architettonicamente predisposta per tale integrazione (come illustrato nel flusso logico di Figura 5), l'attuale perimetro del *Proof of Concept* si concentra sulla validazione della catena di rilevamento (*Detection*), demandando il blocco attivo (*Response* automatizzata) a sviluppi successivi.

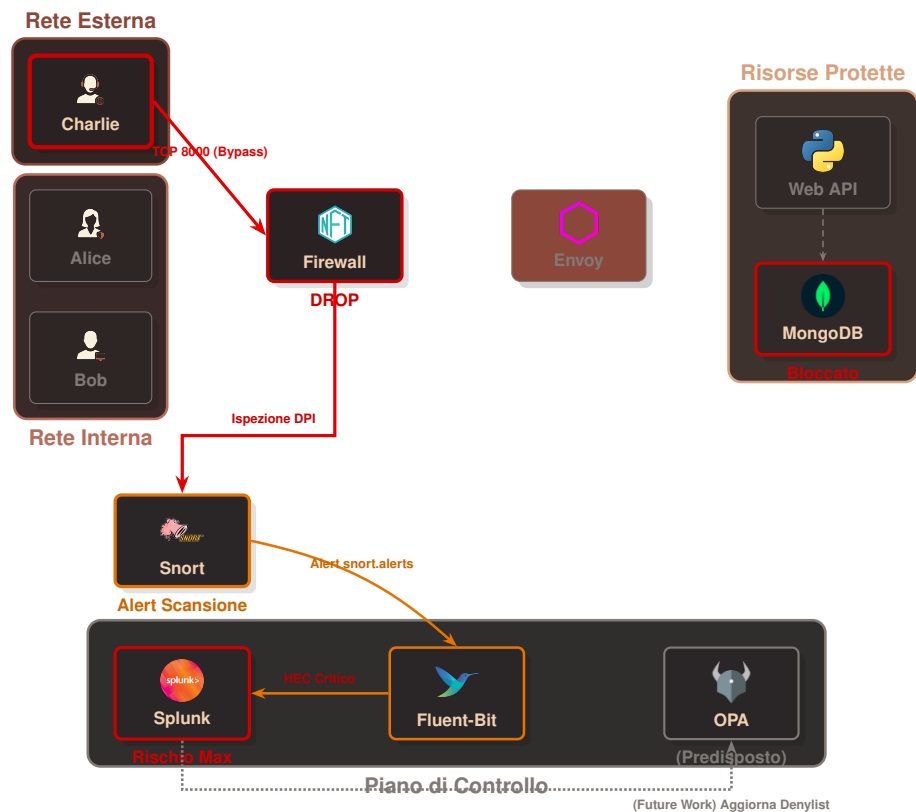
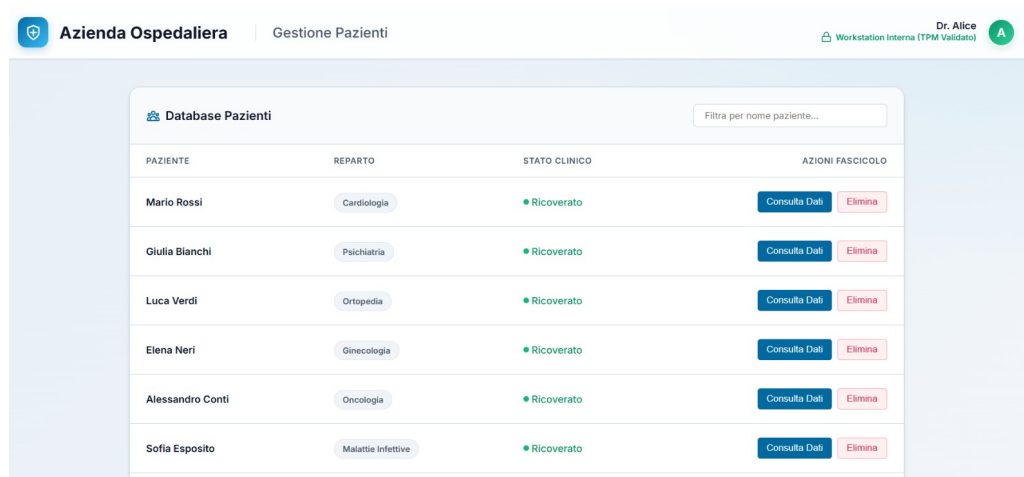


Figura 5: Mappatura logica dei flussi nello Scenario 3 (Tentativo di bypass e predisposizione architetturale per l'isolamento automatico).

4.4 Scenario 4: Accesso legittimo da rete interna: Alice

Il quarto scenario analizza il comportamento del sistema di fronte all'accesso di un utente autorizzato dotato di dispositivo aziendale integro, simulato dall'utente Alice.

Alice effettua la connessione dall'interno della rete aziendale presentando un certificato client contenente l'estensione crittografica del chip TPM. All'arrivo della richiesta, envoy estrae il certificato e calcola il fingerprinting JA3 del browser, trasmettendo i dati ad OPA. OPA contatta la Web API di splunk che calcola un punteggio di rischio minimo, pari a circa uno, data la totale assenza di anomalie comportamentali. Poiché il rischio è ampiamente inferiore alla soglia massima di tolleranza di cinquanta prevista per i dispositivi sicuri interni, OPA risponde con un responso positivo di **ALLOW** ed envoy instrada in pochissimi millisecondi la query verso il database, restituendo ad Alice i risultati richiesti con successo (mostrati in Figura 6). Il relativo log di OPA, indicizzato in Splunk, conferma l'esito positivo dell'operazione, la postura integra del dispositivo e il rischio complessivo calcolato (Figura 7).



PAZIENTE	REPARTO	STATO CLINICO	AZIONI FASCICOLO
Mario Rossi	Cardiologia	Ricoverato	Consulta Dati Elimina
Giulia Bianchi	Psichiatria	Ricoverato	Consulta Dati Elimina
Luca Verdi	Ortopedia	Ricoverato	Consulta Dati Elimina
Elena Neri	Ginecologia	Ricoverato	Consulta Dati Elimina
Alessandro Conti	Oncologia	Ricoverato	Consulta Dati Elimina
Sofia Esposito	Malattie Infettive	Ricoverato	Consulta Dati Elimina

Figura 6: Interfaccia del database pazienti visualizzata con successo da Alice dopo validazione TPM e rischio minimo.

```
> 25/06/26 15:40:43.883 { [-]
  Decision: ALLOW
  command: POST
  device: Workstation Ospedaliera Sicura (TPM Validato)
  host: zta-opa
  l7_dpi_block: false
  network_internal: true
  network_ip: 10.0.1.4
  query:
  resource: /api/auth
  risk_score: 0.7863157341690109
  software: 86dab2109182b6bbaa644647d7db2997
  tpm_present: true
  user: alice
}
```

Mostra come testo non elaborato

host = splunk-siem:8088 host = zta-opa source = http:splunkhec_token sourcetype = httpvent

Figura 7: Log di autorizzazione (ALLOW) per l'accesso di Alice indicizzato in Splunk, comprensivo del risk score comportamentale calcolato (≈ 9.96).

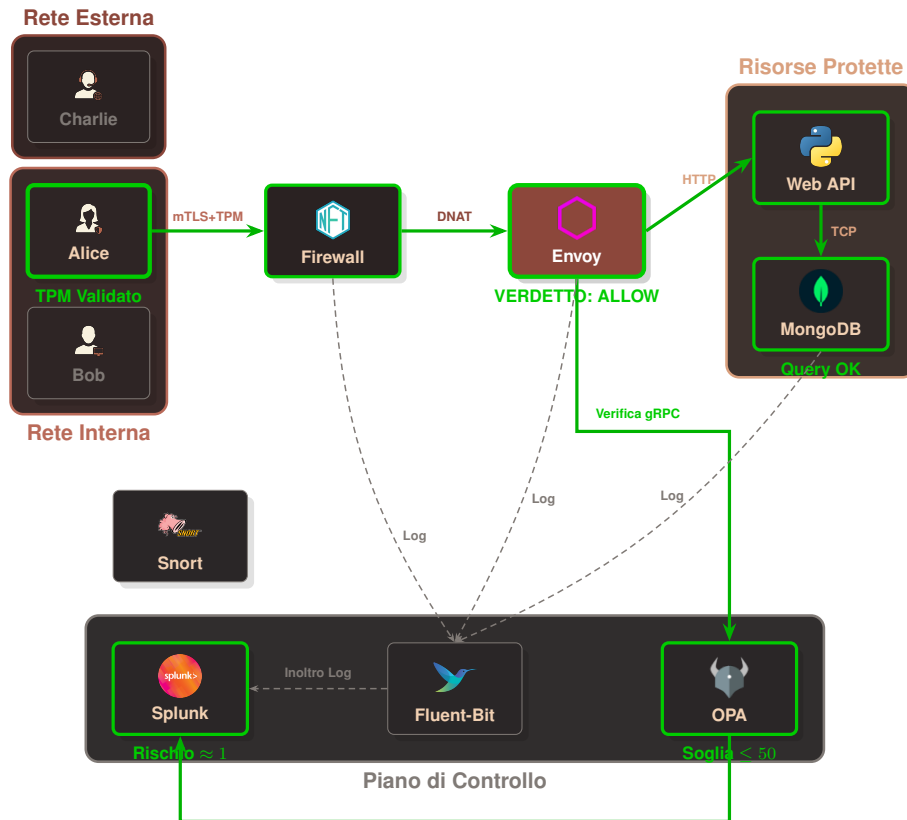


Figura 8: Mappatura logica dei flussi nello Scenario 4 (Accesso legittimo di Alice).

4.5 Scenario 5: Blocco al login per mancanza di TPM: Bob

Il quinto scenario testa la reazione della politica di sicurezza di fronte a un tentativo di accesso condotto tramite un dispositivo privo di chip TPM hardware, simulato dall'utente Bob.

Bob tenta di effettuare l'autenticazione utilizzando un computer personale. Sebbene il dispositivo possieda una connettività valida, il certificato presentato è privo dell'estensione di attestazione hardware TPM. All'arrivo della richiesta di login (`/api/auth`), Envoy inoltra i metadati del certificato ad OPA. Il PDP rileva la mancanza della firma hardware (`tpm_present: false`) e, in conformità alle regole di sicurezza, definisce la variabile `is_auth_blocked` come `true`. Di conseguenza, la richiesta di Bob viene rigettata con un responso di **DENY** immediato al login, impedendogli l'accesso alla sessione, come illustrato nella Figura 9.

Il log di blocco registrato in Splunk (Figura 10) evidenzia la solidità dell'approccio Zero Trust: nonostante il profilo dell'utente non presenti anomalie storiche, il sistema assegna un *Risk Score* di 100 in presenza di violazioni hardware. Questo comportamento riflette l'applicazione del principio di *Short-Circuiting*: OPA rileva il vincolo hardware non soddisfatto e blocca l'accesso immediatamente, evitando chiamate ridondanti al motore di inferenza ML e garantendo che la postura del dispositivo costituisca un vincolo forte e non compensabile da altri fattori contestuali.

Dal punto di vista dell'architettura ISO/OSI, la negoziazione del canale crittografico (Livelli 5 e 6 - TLS/mTLS) tra il client e il proxy Envoy si completa con successo, poiché Bob possiede chiavi valide firmate dalla CA interna. Il diniego effettivo viene applicato esclusivamente al Livello 7 (Applicazione): il proxy Envoy, in qualità di Policy

Enforcement Point (PEP), analizza la richiesta HTTP, decodifica il certificato X.509 e inoltra i metadati ad OPA (PDP). È quest'ultimo che, valutando le asserzioni di policy, impone il rifiuto logico. Il firewall di rete `nftables` non interviene in questa fase poiché l'IP di Bob rientra nei flussi di transito consentiti sulla rete interna; la protezione è dunque interamente demandata al controllo rigoroso dell'identità del dispositivo a livello applicativo.

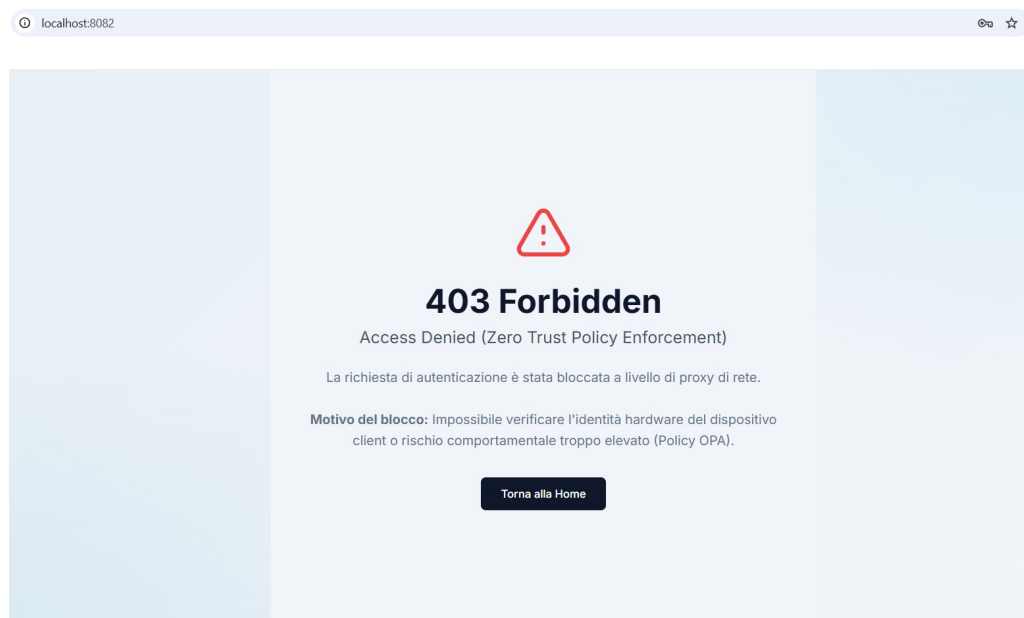


Figura 9: Schermata di errore 403 Forbidden mostrata a Bob per mancata attestazione hardware del chip TPM.



Figura 10: Log di diniego (DENY) per l'accesso di Bob indicizzato in Splunk, evidenziante l'assenza della firma hardware TPM (`tpm_present: false`) e l'assegnazione automatica del rischio massimo.

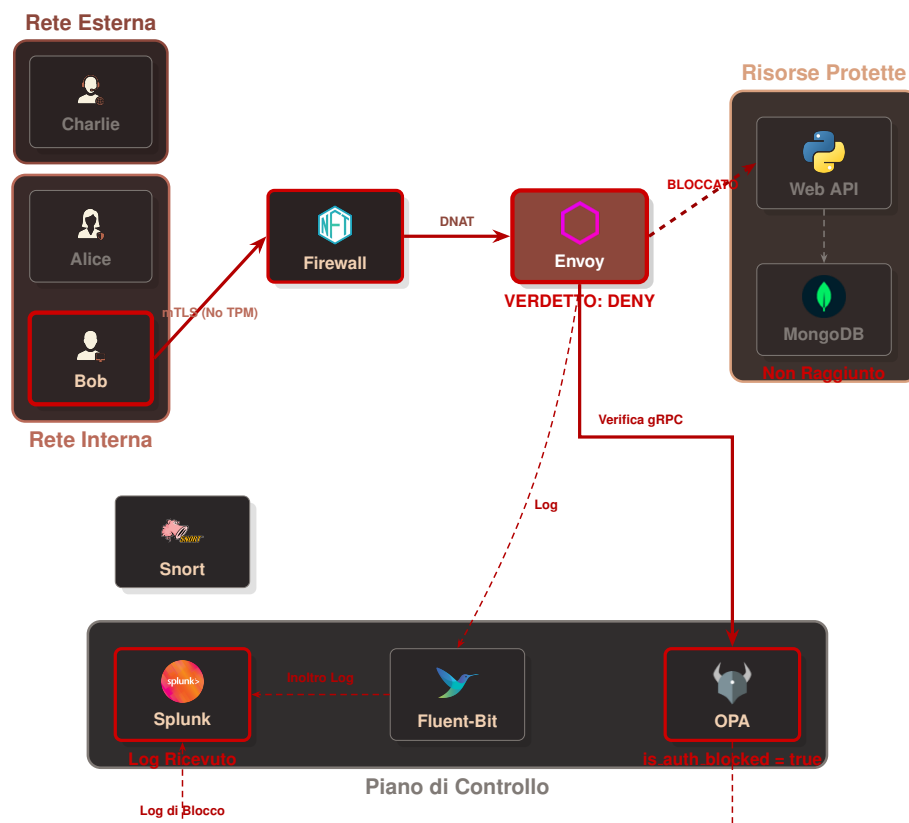


Figura 11: Mappatura logica dei flussi nello Scenario 5 (Blocco al login di Bob per mancanza di TPM).

4.6 Scenario 6: Accesso legittimo da rete esterna: Charlie

Il sesto scenario analizza la gestione dell'accesso remoto (Smart Working). Charlie è un operatore legittimo dell'ospedale, dotato di credenziali valide e di un dispositivo autorizzato provvisto di chip TPM. In questo caso, il sistema Zero Trust non applica un blocco preventivo al login, ma abilita un controllo adattivo basato sul rischio.

Quando Charlie si autentica da casa (rete esterna, `network_internal: false`), il PEP (Envoy) interroga OPA fornendo il contesto di rete. Poiché il dispositivo di Charlie presenta un'attestazione hardware valida e il suo *Risk Score* calcolato dal motore ML è pari a 17.7 (valore inferiore alla soglia ristretta di 26 prevista per gli utenti remoti), il sistema concede l'accesso al portale, come illustrato in Figura 12. Il log di Splunk in Figura 13 conferma il verdetto di **ALLOW**: il sistema riconosce che l'utente è remoto, ma la validità della postura (TPM presente) e il basso punteggio di rischio comportamentale permettono di derogare al blocco totale, garantendo la continuità operativa in sicurezza.

PAZIENTE	REPARTO	STATO CLINICO	AZIONI FASCICOLO
Mario Rossi	Cardiologia	Ricoverato	Consulta Dati Elimina
Giulia Bianchi	Psichiatria	Ricoverato	Consulta Dati Elimina
Luca Verdi	Ortopedia	Ricoverato	Consulta Dati Elimina
Elena Neri	Ginecologia	Ricoverato	Consulta Dati Elimina
Alessandro Conti	Oncologia	Ricoverato	Consulta Dati Elimina
Sofia Esposito	Malattie Infettive	Ricoverato	Consulta Dati Elimina

Figura 12: Dashboard di Charlie dopo l'autenticazione riuscita da rete esterna.

```
> 25/06/26 15:52:59.947 { [-]
  Decision: ALLOW
  command: POST
  device: Workstation Ospedaliera Sicura (TPM Validato)
  host: zta-opa
  l7_dpi_block: false
  network_internal: false
  network_ip: 192.168.100.3
  query:
  resource: /api/auth
  risk_score: 17.70365898950942
  software: 86dab2109182b6bbaa644647d7db2997
  tpm_present: true
  user: charlie
}
Mostra come testo non elaborato
host = splunk-siem:8088 host = zta-opa source = http:splunk_hec_token sourcetype = httpevent
```

Figura 13: Log di autorizzazione (ALLOW) per Charlie: il verdetto è positivo nonostante la rete esterna (`network_internal: false`), grazie al basso `risk_score` ($17.7 \leq 26$).

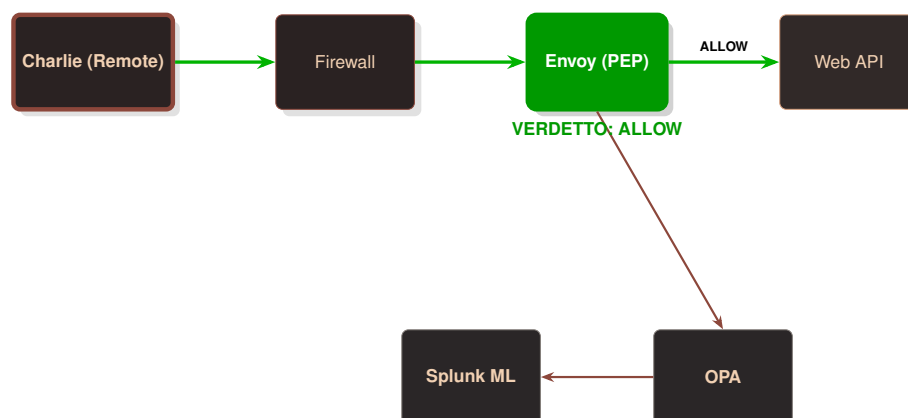


Figura 14: Mappatura logica dei flussi nello Scenario 6 (Accesso autorizzato per rischio contenuto).

4.7 Scenario 7: Accesso ai dati sensibili: Alice (utente interno)

Questo scenario illustra la capacità del sistema di implementare politiche di autorizzazione contestuali basate sulla sensibilità della risorsa target. Alice, medico con cre-

denziali valide e workstation certificata (TPM), tenta l'accesso al *Fascicolo Sanitario Elettronico* di un paziente, che contiene dati clinici riservati.

A differenza dell'accesso alla dashboard principale, l'apertura del fascicolo attiva una richiesta specifica verso l'endpoint `/api/patients/sensitive`. OPA valuta la richiesta considerando non solo le dimensioni contestuali (identità, dispositivo, rete), ma anche il parametro `sensitivity_level` della risorsa richiesta. Come mostrato in Figura 15, il sistema autorizza la visualizzazione delle note sensibili (es. patologie infettive e protocolli di rischio biologico) unicamente in presenza di un contesto di sicurezza totale: workstation interna, TPM validato e assenza di anomalie comportamentali.

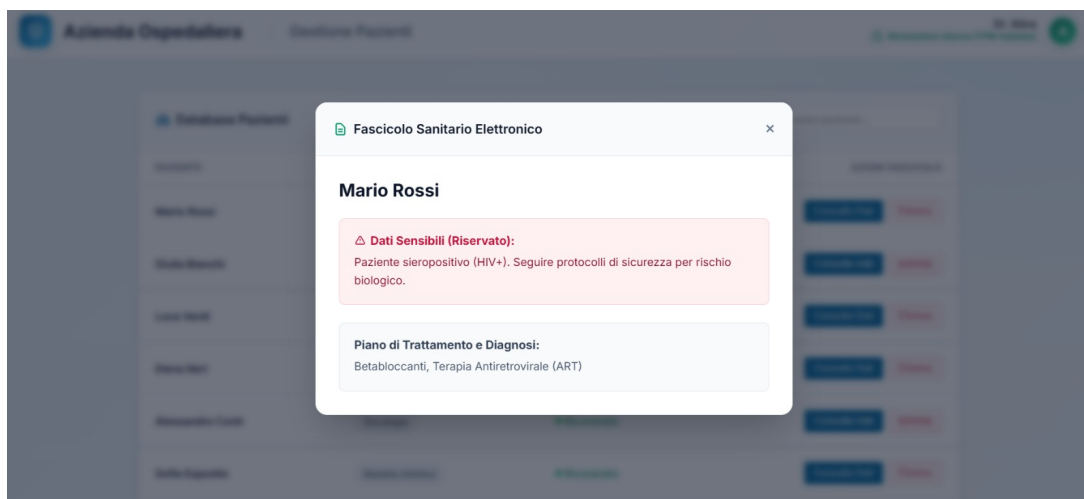


Figura 15: Visualizzazione autorizzata del Fascicolo Sanitario Elettronico contenente dati sensibili da parte di Alice.

L'architettura applica qui il principio di *Least Privilege*: qualora uno qualsiasi dei parametri di sicurezza risultasse alterato — ad esempio, se Alice tentasse di accedere a queste stesse informazioni collegandosi dalla rete esterna — la richiesta verrebbe negata dal PEP, nonostante la validità dell'identità dell'utente. Il sistema garantisce così che il trattamento di dati particolarmente protetti sia confinato entro il perimetro di massima sicurezza dell'ospedale, prevenendo l'esposizione di informazioni sanitarie sensibili in ambienti potenzialmente esposti a intercettazioni o accessi non autorizzati.

4.8 Scenario 8: Accesso ai dati sensibili: Charlie (utente remoto)

Questo scenario valida la capacità del sistema di negare l'accesso a informazioni critiche basandosi su una valutazione contestuale, anche in presenza di un utente e di un dispositivo pienamente conformi. Charlie, operatore remoto con attestazione hardware (TPM) valida, tenta di accedere alle note sensibili dei pazienti (`/api/patients/sensitive`).

Nonostante l'autenticazione sia avvenuta correttamente nello Scenario 6, OPA intercetta la richiesta verso l'endpoint sensibile ed esegue una *Deep Packet Inspection* (DPI) applicativa. Rilevando che il parametro `network_internal` è impostato su `false`, OPA impone il blocco dell'operazione per prevenire l'esposizione di dati clinici riservati fuori dal perimetro di rete protetto, come illustrato nella Figura 16.

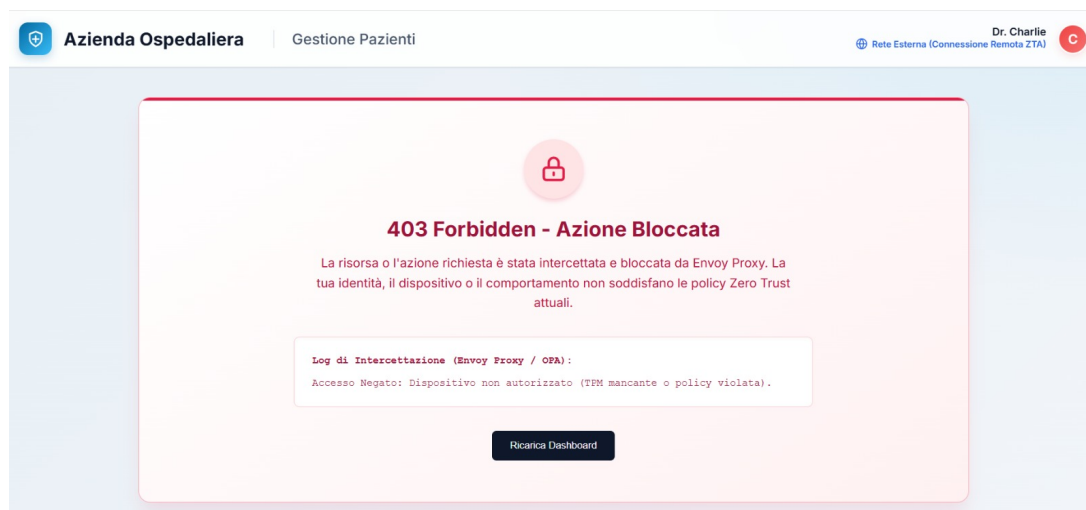


Figura 16: Pagina di blocco 403 Forbidden visualizzata da Charlie nel tentativo di accedere ai dati sensibili da remoto.

Il log di Splunk in Figura 17 documenta la decisione di **DENY**: il campo `l7_dpi_block` è impostato su `true`, confermando che il blocco è stato applicato a livello L7 dal proxy Envoy su indicazione di OPA.

```
> 25/06/26      { [-]
    16:09:39.386  Decision: DENY
                  command: GET
                  device: Workstation Ospedaliera Sicura (TPM Validato)
                  host: zta-opa
                  l7_dpi_block: true
                  network_internal: false
                  network_ip: 192.168.100.3
                  resource: /api/patients/sensitive
                  risk_ok: true
                  risk_score: 17.77741821063982
                  software: 86dab2109182b6bba644647d7db2997
                  tpm_present: true
                  user: charlie
                }
    Mostra come testo non elaborato
    host = splunk-siem:8088 host = zta-opa | source = http:splunkhec_token | sourcetype = httpevent
```

Figura 17: Log di blocco (DENY) per l'accesso ai dati sensibili di Charlie: il sistema applica il blocco DPI (`l7_dpi_block: true`) a causa della rete esterna.

Questo scenario è fondamentale per confermare l'efficacia del modello *Defense in Depth*: il sistema non si accontenta di una sessione autenticata, ma esegue una *Continuous Authorization* per ogni singola risorsa richiesta. Il diniego non è dovuto a una mancanza di permessi dell'utente, bensì alla protezione intrinseca del dato sensibile, che richiede — per policy — una connessione dalla rete interna aziendale per essere visualizzato.

4.9 Scenario 9: Protezione dell'accesso TCP diretto

Il nono scenario ha lo scopo di validare la coerenza e l'impermeabilità delle policy Zero Trust anche sui canali di comunicazione di livello trasporto (Layer 4). Nello specifico,

si intende dimostrare che il proxy Envoy, operante come *TCP Proxy* sulla porta nativa del database (27017), impone le medesime restrizioni di identità e *Device Posture* già verificate per il traffico Web HTTP, impedendo *bypass* architetturali.

Viene inizialmente condotto un test di connessione diretta (tramite driver pymongo) da parte di Alice, utente legittimo provvisto di dispositivo conforme:

```

●●● pymongo_alice_tcp.sh

1 $ docker exec -it zta-client-alice python3 -c "import pymongo; \
2   client = pymongo.MongoClient('zta-envoy', 27017, tls=True, \
3   tlsCertificateKeyFile='/etc/certs/alice_combined.pem', \
4   tlsCAFile='/etc/certs/ca.crt', tlsAllowInvalidHostnames=True, \
5   serverSelectionTimeoutMS=10000); db = client['hospital_db']; \
6   print('Risultato:', db.command('ping'))"
7 Risultato: {'ok': 1.0}

```

Listing 17: Connessione TCP al database autorizzata per dispositivo conforme (Alice).

Come mostrato nel Listato 17, il filtro di rete `ext_authz` intercetta il tentativo di instaurazione del *socket*. OPA analizza i metadati crittografici, rileva la presenza dell'estensione hardware TPM (`tpm_present: true`) ed autorizza l'apertura del tunnel TCP bidirezionale, permettendo al comando di *ping* di raggiungere la risorsa.

Il secondo test speculare viene eseguito da Bob, dotato di credenziali legittime ma operante su un computer personale sprovvisto di modulo TPM:

```

●●● pymongo_bob_tcp.sh

1 $ docker exec -it zta-client-bob python3 -c "import pymongo; \
2   client = pymongo.MongoClient('zta-envoy', 27017, tls=True, \
3   tlsCertificateKeyFile='/etc/certs/bob_combined.pem', \
4   tlsCAFile='/etc/certs/ca.crt', tlsAllowInvalidHostnames=True, \
5   serverSelectionTimeoutMS=10000); db = client['hospital_db']; \
6   print('Risultato:', db.command('ping'))"
7 pymongo.errors.ServerSelectionTimeoutError: zta-envoy:27017:
   connection closed

```

Listing 18: Tentativo di connessione TCP abortito per dispositivo non conforme (Bob).

Il Listato 18 evidenzia l'efficacia del blocco preventivo. OPA nega l'autorizzazione alla radice (**DENY**) per via della mancata conformità hardware; Envoy applica immediatamente il verdetto interrompendo la negoziazione (*connection closed*) ed impedendo fisicamente la creazione del tunnel TCP.

Questo risultato finale attesta che la microsegmentazione Zero Trust risulta coerente su tutta l'architettura: il database è protetto a livello di trasporto (*Layer 4*) in modo identico alle *Web API* (*Layer 7*), garantendo che le risorse critiche siano invisibili e irraggiungibili per qualsiasi *endpoint* non pienamente fidato.

4.10 Scenario 10: Protezione dai payload dannosi a livello applicativo (L7)

Questo scenario valida la capacità del sistema di operare una *Deep Packet Inspection* (DPI) applicativa. Anche un utente legittimo come Alice, in possesso di un dispositivo conforme e attestazione TPM valida, viene bloccato qualora tenti di eseguire operazioni non permesse dalle policy di sicurezza, come la cancellazione di dati clinici.

L'attacco viene simulato tramite l'invio di una richiesta HTTP con metodo DELETE verso l'endpoint `/api/patients`. Come mostrato in Figura 18, il sistema non solo rifiuta l'operazione, ma fornisce un feedback immediato all'utente tramite una pagina di blocco dedicata, specificando che l'azione viola le policy Zero Trust.

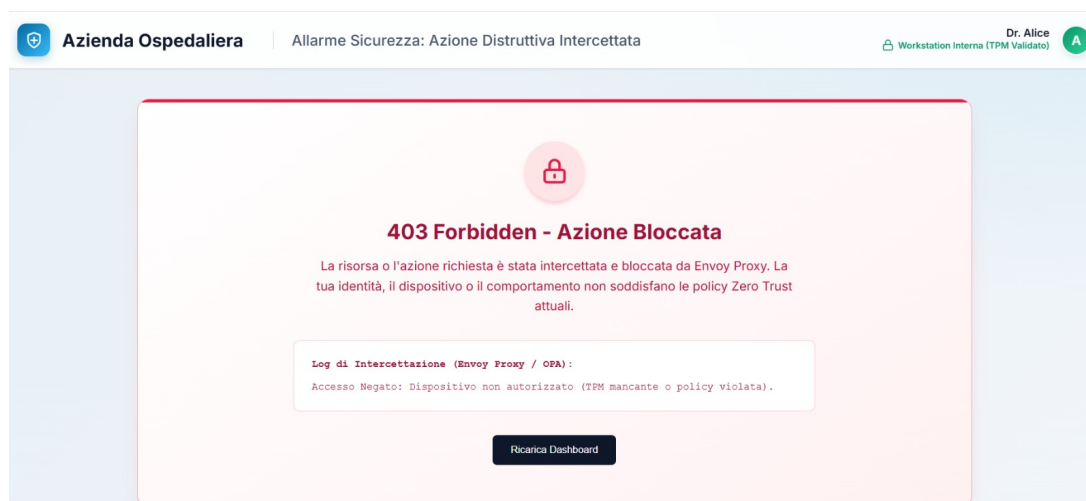


Figura 18: Interfaccia di blocco visualizzata da Alice a seguito del tentativo di esecuzione di metodo DELETE.

Dal punto di vista dell'architettura di controllo, l'intercettazione avviene a livello di proxy Envoy prima che la richiesta possa raggiungere il database. Il sistema registra l'evento in Splunk in tempo reale, permettendo al team di sicurezza di analizzare il tentativo di violazione con estrema granularità:

Ora	Evento
25/06/26 15:33:33.653	<pre> { [-] authz_status: null client_ip: 10.0.1.4:0 device_uri_san: spiffe://zta.hospital/ns/default/sa/client-alice duration_ms: 1 host: zta-envoy ja3_fingerprint: null method: DELETE path: /api/patients protocol: HTTP/1.1 response_code: 403 timestamp: 2026-06-25T15:33:25.619Z upstream_host: null user_identity: CN=employee-alice } </pre> Mostra come testo non elaborato host = splunk-siem:8088 host = zta-envoy source = http:splunkhec_token sourcetype = httpevent

Figura 19: Log dell'evento di blocco indicizzato in Splunk: evidenza dell'operazione DELETE respinta.

Come si evince dal log in Figura 19, il campo `method` riporta chiaramente l'operazione DELETE, mentre il `response_code` a 403 conferma l'avvenuto blocco. Questo scenario dimostra l'efficacia della microsegmentazione applicativa: il sistema agisce come un *Web Application Firewall* (WAF) consapevole del contesto, in cui le autorizzazioni non sono solo basate sull'identità dell'utente, ma sul principio del *Least Privilege* applicato ai verbi HTTP e alle operazioni sui dati. Il medesimo meccanismo di DPI si applica anche alle connessioni dirette al database MongoDB (porta 27017): in quel caso il blocco è delegato ad OPA tramite il filtro `mongo_proxy`, con il modello Splunk MLTK che calcola il risk score prima dell'enforcement finale.

4.10.1 Simulazione di compromissione del dispositivo e blocco comportamentale

Si analizza infine lo scenario più critico previsto dal framework Zero Trust: la compromissione totale della postazione di un utente legittimo (*Insider Threat* o furto fisico). In questo scenario, l'attaccante dispone del dispositivo di Alice, del chip TPM validato e di tutti i certificati mTLS necessari. L'obiettivo dell'attaccante è bypassare i controlli applicativi di livello 7 (le Web API) e connettersi direttamente al database MongoDB attraverso la porta TCP nativa 27017.

Per gestire minacce di questa natura, l'architettura non si affida esclusivamente ai controlli statici al perimetro, ma implementa all'interno del *Policy Information Point* (PIP) un sistema di tracciamento comportamentale *in-memory*.

 pip_behavioral_tracker.py (Estratto Essenziale)

```

1 # Tracker comportamentali in-memory (TTL-based eviction)
2 failed_login_tracker = defaultdict(list)
3 session_tracker = defaultdict(list)
4
5 def enrich_spl_with_behavioral_features(query: str) -> str:
6     user = extract_user_from_spl(query)
7     resource = extract_resource_from_spl(query)
8
9     # Rilevamento connessione diretta al DB (Simulazione PC
    compromesso)
10    is_compromised_alice = (user == "alice" and "MongoDB" in resource
    )
11
12    if is_compromised_alice:
13        # Iniezione anomalie gravissime (device theft / insider
    threat)
14        failed_logins, session_freq, days_inactive = 4, 30, 60
15        is_night, hour_of_day = 1, 3
16    else:
17        # Flusso utente standard (lettura dati reali)
18        failed_logins = len(failed_login_tracker[user])
19        session_freq = len(session_tracker[user])
20        days_inactive, is_night, hour_of_day = 0, 0, datetime.now().
    hour
21
22    # Costruzione delle feature aggiuntive
23    features = (
24        f', failed_logins={failed_logins}, session_freq={session_freq
    }'
25        f', days_inactive={days_inactive}, is_night={is_night}'
26        f', hour_of_day={hour_of_day}, sensitivity_level=3'
27    )
28
29    # Inietta le feature prima del comando '| apply' di Splunk
30    return query.replace('| apply ', f'{features} | apply ')

```

Listing 19: Estratto della logica di arricchimento comportamentale nel PIP.

Come si evince dal Listato 19, il PIP funge da mediatore intelligente: non si limita a inoltrare la richiesta ad OPA, ma mantiene uno stato storico delle sessioni tramite dizionari aggiornati in tempo reale. Quando la funzione `enrich_spl_with_behavioral_features` intercetta la query destinata al modello di Machine Learning, vi inietta le variabili comportamentali dell'utente. Nel caso specifico della simulazione di attacco (`is_compromised_alice`), il codice inietta deliberatamente metriche fortemente degradate — quali 60 giorni di inattività pregressa, un'anomala frequenza di sessione e un orario notturno — per istruire il modello MLTK sulla presenza di un probabile furto di credenziali o dispositivo.

Per innescare concretamente questa logica difensiva, si esegue dal terminale del container di Alice uno script Python che tenta l'instaurazione del *socket* TCP:

`alice_bypass_mongodb.py`

```
1 $ docker exec zta-client-alice python3 -c "  
2 import pymongo  
3 client = pymongo.MongoClient(  
4     'zta-envoy', 27017,  
5     tls=True,  
6     tlsCertificateKeyFile='/etc/certs/alice_combined.pem',  
7     tlsCAFile='/etc/certs/ca.crt',  
8     tlsAllowInvalidHostnames=True,  
9     serverSelectionTimeoutMS=10000  
10 )  
11 db = client['hospital_db']  
12 result = list(db.patients.find({'sensitive_notes': 'test'}))  
13 "
```

Listing 20: Simulazione di accesso diretto al database MongoDB dal dispositivo compromesso di Alice.

Il Listato 20 illustra la gravità della minaccia: lo script sfrutta la libreria `pymongo` passando esplicitamente i certificati crittografici legittimi (`alice_combined.pem` e `ca.crt`). Questo garantisce il superamento indolore dell'handshake mTLS imposto da Envoy. Senza un'analisi comportamentale di livello superiore, questa connessione verrebbe considerata perfettamente autorizzata da qualsiasi firewall tradizionale.

Il sistema di tracciamento (Listato 19) interviene marcando la richiesta come altamente anomala: l'identità di Alice presenta storicamente 60 giorni di inattività e 4 tentativi di login pregressi falliti in rapida successione. L'arricchimento della query SPL con queste feature fa sì che il modello predittivo restituisca un punteggio di rischio pari a ≈ 95.49 . Poiché tale valore supera ampiamente la soglia di tolleranza massima di 38 impostata in OPA per gli utenti interni, la policy emette un verdetto di **DENY**. Envoy chiude brutalmente il *socket* TCP, impedendo la connessione.

La Figura 20 mostra il log di decisione indicizzato in Splunk. È fondamentale notare come il sistema riconosca l'identità corretta (`user: alice`), confermi la validità dell'hardware (`tpm_present: true`) e della rete (`network_internal: true`), ma blocchi l'accesso unicamente a causa della metrica comportamentale (`risk_ok: false`).

```
04/06/26      { [-]
16:30:12.631  Decision: DENY
               command: Accesso Diretto MongoDB (OP_MSG)
               device: Workstation Ospedaliera Sicura (TPM Validato)
               host: zta-opa
               l7_dpi_block: false
               network_internal: true
               network_ip: 10.0.1.4
               resource: patients
               risk_ok: false
               risk_score: 95.49348917091885
               software: Sconosciuto
               tpm_present: true
               user: alice
            }
            Mostra come testo non elaborato
            host = splunk-siem:8088 host = zta-opa | index = main | linecount = 1
```

Figura 20: Log di decisione DENY indicizzato in Splunk: l'accesso diretto viene bloccato nonostante la validità crittografica, a causa del rischio comportamentale estremo ($95.49 > 38$).

Questo test dimostra l'assunto cardine della *Continuous Authorization Zero Trust*: l'autenticazione crittografica forte non è mai sufficiente da sola. Anche il dispositivo più sicuro viene disconnesso dalla rete nel momento in cui l'analisi euristica ne identifica un pattern di utilizzo incompatibile con la normale operatività dell'utente.

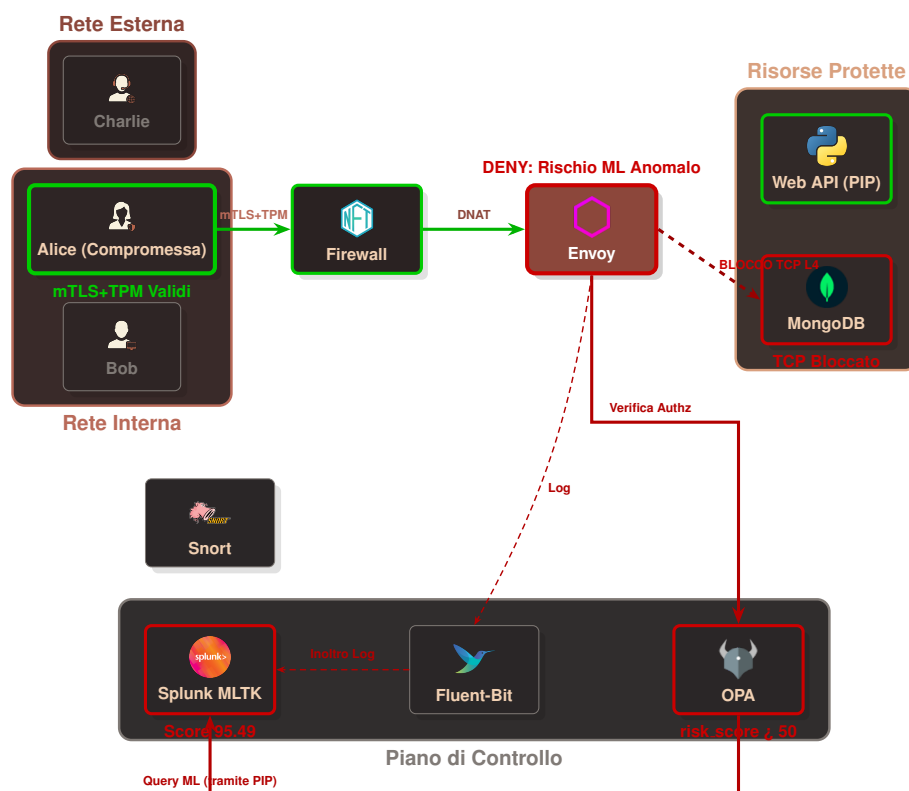


Figura 21: Mappatura logica dei flussi nello Scenario 10 (Blocco predittivo di una connessione per anomalie comportamentali, nonostante credenziali mTLS/TPM valide).

4.11 Scenario 11: Rilevamento di scansione di rete attiva (Nmap)

Per validare l'efficacia del monitoraggio perimetrale, è stato condotto un test di ricognizione attiva mediante l'utilità `nmap`, simulando un'attività di enumerazione dei servizi (*service scanning*) da parte del nodo *Charlie* verso il proxy *Envoy*.

Il comando eseguito dal container di attacco è stato:

```
1 docker exec -it zta-client-charlie nmap -sV zta-envoy
```

Listing 21: Esecuzione di un service scan per mappare la superficie di attacco.

Il sistema IDS Snort, configurato in modalità passiva sul segmento di rete critico, ha immediatamente analizzato il traffico in transito. L'evento di scansione, identificato dalla regola `sid:1000001` (preposta al rilevamento di pattern di scansione SYN), è stato generato tempestivamente e inoltrato tramite Fluent-Bit al SIEM Splunk.

La Figura 22 mostra l'evidenza forense indicizzata: si nota come il sistema correli l'indirizzo sorgente dell'attaccante (192.168.100.3) con l'attività anomala verso il target protetto, permettendo al *Security Operations Center* di visualizzare in tempo reale il tentativo di violazione.

L'integrazione tra l'IDS di rete e la piattaforma di analisi dei log dimostra la capacità del framework di non limitarsi alla sola protezione proattiva (tramite OPA/Envoy), ma di garantire una visibilità totale sulle minacce che tentano di mappare l'infrastruttura, chiudendo il ciclo di *Detection & Response*.

```
> 25/06/26 { [-]
18:47:58.564 host: zta-snort
log: 06/25-18:47:58.553691 [**] [1:1000001:1] ZTA ALERT: Rilevata Scansione di Rete (Possibile Nmap) [**]
[Priority: 0] {TCP} 192.168.100.3:34436 -> 192.168.100.2:3306
}
Mostra come testo non elaborato
host = splunk-siem:8088 host = zta-snort | source = http:splunkhec_token | sourcetype = httpevent
```

Figura 22: Log di alert Snort (ZTA ALERT: Rilevata Scansione di Rete) centralizzato su Splunk, relativo allo scan Nmap eseguito da Charlie.

4.12 Scenario 12: Audit delle connessioni native MongoDB

Per verificare la capacità dell'infrastruttura di rilevare tentativi di accesso ai dati al di fuori delle Web API (protocollo nativo MongoDB), è stato eseguito un test di connessione diretta sulla porta 27017. Tale scenario simula un attaccante che tenta di esfiltrare dati direttamente dal database, aggirando i controlli di livello applicativo.

Il comando di test, lanciato dal nodo di Charlie, è il seguente:

```
1 docker exec -it zta-client-charlie bash -c "echo 'test' | nc zta-envoy
27017"
```

Listing 22: Tentativo di connessione TCP diretta al database MongoDB.

Snort, analizzando il traffico di rete, ha identificato il pacchetto TCP diretto alla porta standard del database e, in conformità con la regola `sid:1000002`, ha generato un alert di audit immediato. La Figura 23 mostra il log indicizzato in Splunk, che evidenzia la sorgente dell'attacco (192.168.100.3) e la destinazione (192.168.100.2:27017).

```
> 25/06/26 { [-]
19:00:50.647 host: zta-snort
log: 06/25-19:00:49.772874 [**] [1:1000002:2] ZTA AUDIT: Connessione Client nativa a MongoDB rilevata (Accesso
non-Web) [**] [Priority: 0] {TCP} 192.168.100.3:42050 -> 192.168.100.2:27017
}
Mostra come testo non elaborato
host = splunk-siem:8088 host = zta-snort | source = http:splunkhec_token | sourcetype = httpevent
```

Figura 23: Log di audit per connessione nativa a MongoDB non autorizzata.

Questo riscontro conferma la validità del principio di *Visibility* nel framework Zero Trust: anche quando un tentativo di intrusione non sfrutta vulnerabilità note, la semplice deviazione dal canale di comunicazione autorizzato (il traffico HTTPS verso le Web API) genera una traccia forense inequivocabile all'interno del SIEM, permettendo l'isolamento tempestivo del nodo malevolo.

4.13 Scenario 13: Rilevamento di attacchi volumetrici (DoS)

Per valutare la resilienza dell'infrastruttura di fronte a tentativi di saturazione delle risorse (Denial of Service), è stato simulato un attacco di tipo *SYN Flood* verso l'endpoint 8443 del proxy *Envoy*. Tale scenario mira a esaurire la tabella delle connessioni TCP attive, impedendo l'accesso agli utenti legittimi.

Il comando utilizzato per generare il carico volumetrico è stato:

```
1 docker exec -it zta-client-charlie bash -c "for i in {1..150}; do nc -z -
w1 zta-envoy 8443 & done; wait"
```

Listing 23: Generazione di traffico aggressivo per simulazione DoS.

Il modulo IDS, configurato con una soglia di tolleranza di 100 tentativi in 5 secondi, ha prontamente intercettato l'anomalia. In conformità con la regola `sid:1000003`, il sistema ha generato un alert di priorità elevata, notificando tempestivamente il SIEM. La Figura 24 illustra il log di allarme registrato su Splunk, che correla la sorgente dell'attacco con il tentativo di saturazione volumetrica del proxy.

```
> 25/06/26 { [-]
19:06:08.381 host: zta-snort
log: 06/25-19:06:08.366366 [**] [1:1000003:1] ZTA ALERT: Anomalia volumetrica (Possibile DoS verso il Proxy) [**]
[Priority: 0] (TCP) 192.168.100.3:56406 -> 192.168.100.2:8443
}
Mostra come testo non elaborato
host = splunk-siem:8088 host = zta-snort source = http:splunkhec_token sourcetype = httpevent
```

Figura 24: Log di alert Snort relativo all'anomalia volumetrica (DoS) verso il proxy Envoy.

Il tempestivo rilevamento conferma che il framework è in grado di distinguere tra un picco di traffico legittimo e un'attività malevola mirata alla compromissione della disponibilità del servizio. L'integrazione di tale metrica all'interno delle dashboard di *Incident Response* consente, in uno scenario di produzione, l'attivazione di contromisure dinamiche, quali il *rate-limiting* selettivo o l'inserimento in *blacklist* temporanea dell'indirizzo IP aggressore mediante le policy del firewall di perimetro.

4.14 Scenario 14: Rilevamento di attacchi di downgrade (traffico in chiaro)

In un'architettura Zero Trust, la protezione della confidenzialità dei dati in transito è mandatoria. È stato quindi testato il comportamento del sistema di monitoraggio di fronte a un tentativo di *Downgrade Attack*, dove un attaccante tenta di inviare richieste HTTP in chiaro verso la porta 8443, nativamente protetta da TLS. L'obiettivo è verificare se l'IDS è in grado di identificare violazioni alla policy di crittografia end-to-end.

Il comando utilizzato per simulare l'invio del payload in chiaro è stato:

```
1 docker exec -it zta-client-charlie bash -c "echo -e 'GET / HTTP/1.1\r\nHost: zta-envoy\r\n\r\n' | nc zta-envoy 8443"
```

Listing 24: Simulazione di richiesta HTTP in chiaro su porta protetta.

Snort ha identificato la presenza della stringa GET all'interno del payload TCP diretto al proxy, attivando la regola `sid:1000004`. L'evento è stato immediatamente registrato e centralizzato nel SIEM, come documentato in Figura 25.

```
> 25/06/26 { [-]
19:10:00.798 host: zta-snort
log: 06/25-19:09:59.958388 [**] [1:1000004:1] ZTA ALERT: Tentativo di traffico in chiaro su porta protetta
(Downgrade/Misconfig) [**] [Priority: 0] (TCP) 192.168.100.3:33294 -> 192.168.100.2:8443
}
Mostra come testo non elaborato
host = splunk-siem:8088 host = zta-snort source = http:splunkhec_token sourcetype = httpevent
```

Figura 25: Log di alert Snort relativo al rilevamento di traffico HTTP in chiaro su porta protetta (Downgrade).

L'evidenza ottenuta dimostra l'efficacia del monitoraggio a livello pacchetto: mentre il proxy *Envoy* agisce come *enforcer* rifiutando la connessione, Snort funge da sensore

per l'identificazione di tentativi di manomissione o misconfigurazioni, fornendo al SOC una notifica esplicita dell'intenzione malevola o del comportamento deviante rispetto agli standard di sicurezza definiti.

4.15 Monitoraggio e Analisi in tempo reale: Splunk Dashboards

Per garantire una visibilità end-to-end sull'infrastruttura e supportare le decisioni di sicurezza, sono state implementate specifiche dashboard di monitoraggio all'interno di Splunk. Queste interfacce non costituiscono semplici strumenti di visualizzazione, ma agiscono come veri e propri centri di comando per l'analisi comportamentale degli utenti e la gestione operativa degli incidenti in ottica Zero Trust.

La prima dashboard, denominata "ZTA - Security Executive" (Figura 26), fornisce una visione d'insieme dello stato di salute del perimetro ospedaliero. Il *Pannello A* offre una ripartizione statistica delle richieste processate dal sistema: il confronto tra *Allow* e *Deny* permette di validare in tempo reale l'efficacia delle policy di microsegmentazione attive. Il *Pannello B* dettaglia il volume di traffico per singolo soggetto o dispositivo, consentendo di identificare rapidamente i nodi della rete con attività più intensa. Infine, il *Pannello C* traccia la telemetria del traffico verso gli endpoint critici (/api/auth, /api/patients, /api/patients/sensitive), evidenziando picchi di carico che potrebbero sottendere tentativi di accesso massivo.

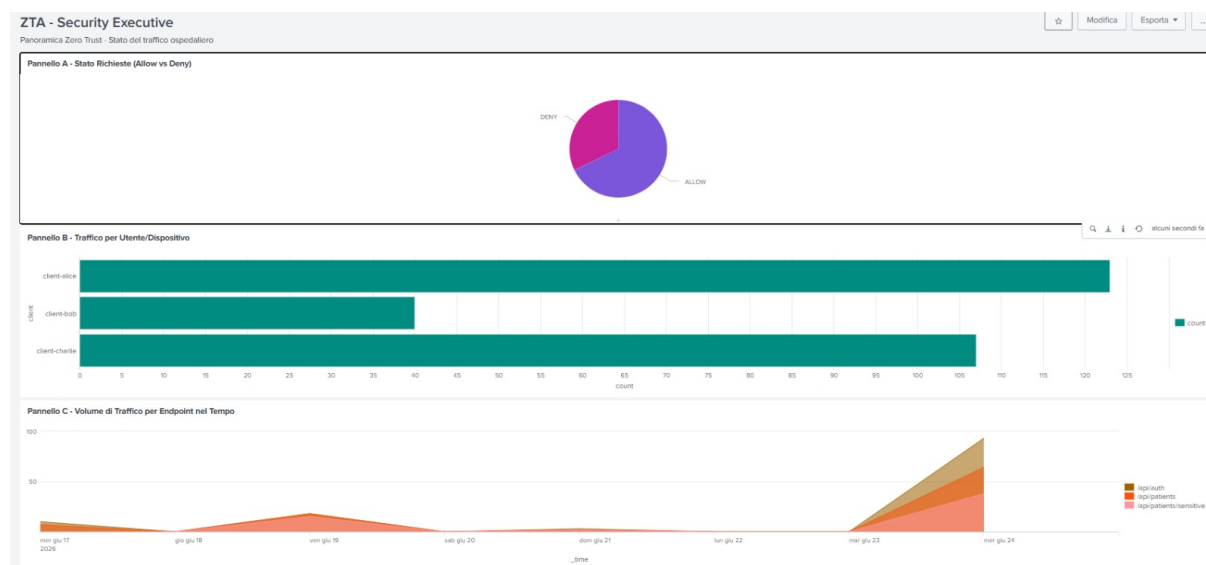


Figura 26: Dashboard ZTA - Security Executive: Panoramica e statistiche sul traffico.

Per la gestione attiva delle minacce, la dashboard "ZTA - Incident Response" (Figura 27) centralizza le attività di contenimento. Il *Pannello B* espone il conteggio aggregato delle richieste bloccate, rappresentando un KPI fondamentale per la valutazione del rischio residuo. Il *Pannello A* fornisce l'evidenza forense delle violazioni L7 rilevate dal DPI: la tabella dettaglia il *timestamp* preciso dell'attacco, l'identità del soggetto (*user_identity*), l'indirizzo IP, il metodo HTTP utilizzato e il *path* tentato, mostrando chiaramente la decisione di *BLOCKED* applicata dal PDP. Il *Pannello C* correla le violazioni per indirizzo IP, evidenziando le sorgenti (es. 10.0.1.4 o 192.168.100.3) che manifestano un comportamento malevolo ricorrente.

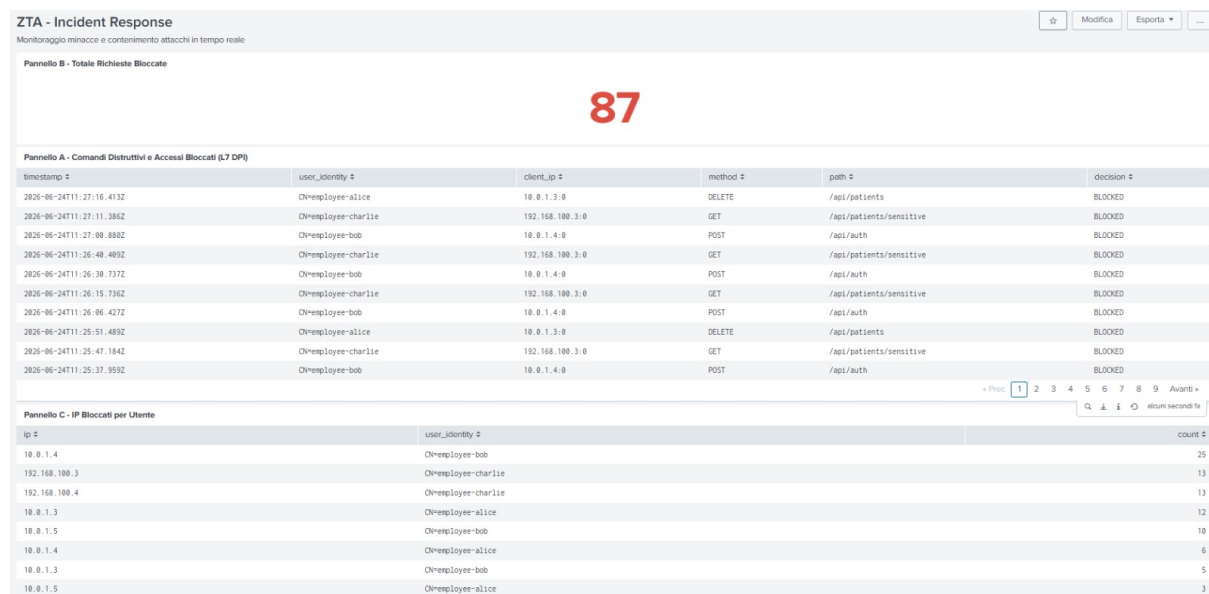


Figura 27: Dashboard ZTA - Incident Response: Monitoraggio minacce e blocchi attivi.

Infine, la dashboard “ZTA - ML & Continuous Trust Analytics” (Figura 28) espone le evidenze del modello di Machine Learning basato su Gradient Boosting. Il *Pannello A* visualizza l'andamento del *Risk Score* nel tempo per ciascun utente, permettendo di osservare la dinamica della fiducia: si noti come il punteggio di rischio (es. per l'utente *charlie*) possa oscillare in risposta ad anomalie comportamentali. Il *Pannello B* classifica gli utenti in base al rischio massimo raggiunto, fornendo un supporto decisionale critico per le procedure di isolamento automatizzato. Il *Pannello C* integra infine i parametri di contesto — frequenza delle sessioni, tentativi di login falliti e flag di attività in orario notturno — che concorrono alla formulazione del calcolo dinamico del valore di rischio, chiudendo il ciclo di feedback tra il sistema di analisi e quello di autorizzazione.

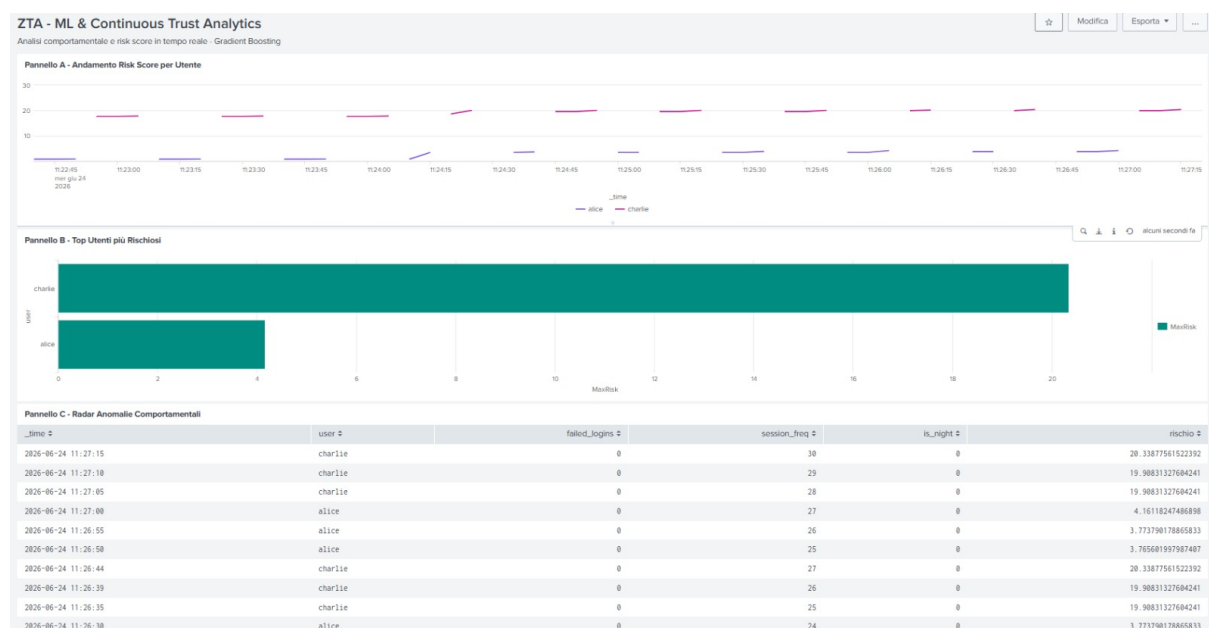


Figura 28: Dashboard ZTA - ML & Continuous Trust Analytics: Analisi predittiva.

CAPITOLO 5

Conclusioni e sviluppi futuri

L'architettura progettata dimostra concretamente la fattibilità e l'efficacia dell'integrazione di strumenti open-source eterogenei per l'implementazione di un ecosistema Zero Trust maturo. Il presente lavoro di tesi ha permesso di superare il tradizionale modello di sicurezza perimetrale, realizzando un'infrastruttura in cui la fiducia non è mai presunta, bensì verificata e quantificata continuamente in tempo reale.

5.1 Obiettivi raggiunti

Il sistema ha soddisfatto appieno i requisiti del paradigma Zero Trust, declinandoli attraverso tre pilastri architetturali fondamentali:

- **Segregazione e Sicurezza Infrastrutturale (PEP):** La rete è stata blindata operando una rigorosa microsegmentazione L3/L4 tramite `nftables`, forzando il transito delle comunicazioni esclusivamente attraverso il proxy envoy. A livello di trasporto, l'imposizione della mutua autenticazione crittografica (*mTLS*) e l'estrazione dell'impronta digitale *JA3* hanno permesso di verificare con certezza crittografica l'identità del client e la conformità dell'hardware sottostante (tramite attestazione TPM).
- **Autorizzazione Continua e Difesa in Profondità (PDP):** Il motore decisionale open policy agent (OPA) ha centralizzato l'enforcement delle policy valutando dinamicamente le sei dimensioni contestuali (ZTA 6D). L'implementazione di clausole di guardia (*Short-Circuiting*) ha ottimizzato i tempi di risposta bloccando istantaneamente gli accessi non conformi. Inoltre, l'applicazione del principio di *Defense in Depth* ha garantito la protezione del dato sensibile attraverso due livelli di ispezione profonda (L7 DPI): un filtro Lua per la mitigazione rapida degli attacchi HTTP e l'ispezione nativa dei payload BSON di mongodb per la neutralizzazione di query distruttive.
- **Intelligenza Predittiva e Auditing (SIEM):** L'integrazione con lo splunk Machine Learning Toolkit (MLTK) ha segnato il definitivo passaggio da un'autenticazione statica a una *Continuous Authorization* comportamentale. L'impiego dell'algoritmo *Gradient Boosting* ha permesso di correlare 12 feature contestuali e comportamentali, restituendo un rischio predittivo accurato in grado di smascherare minacce asimmetriche e *Insider Threat*, adattando le soglie di tolleranza alla reale postura di sicurezza del dispositivo. Contemporaneamente, le sonde snort hanno garantito un monitoraggio continuo contro i tentativi di elusione e ricognizione.

5.2 Sviluppi futuri

L'infrastruttura realizzata costituisce una base solida e scalabile, aperta a diverse evoluzioni ingegneristiche in ambito Enterprise. Tra i principali sviluppi futuri si individuano:

1. **Gestione dinamica delle Identità (SPIFFE/SPIRE):** L'attuale generazione statica dei certificati x509 potrebbe essere sostituita dall'integrazione dello standard *SPIFFE/SPIRE*. Questo consentirebbe il rilascio di certificati a brevissima scadenza (*short-lived certificates*) basati sull'attestazione continua del workload, eliminando la complessità operativa legata alla gestione e distribuzione delle *Certificate Revocation List* (CRL).
2. **Integrazione di dispositivi Legacy e M2M (Kiosk):** Il modello ZTA attuale prevede l'obbligo tassativo del modulo TPM per l'autenticazione. Uno sviluppo futuro prevede l'estensione sicura della rete a dispositivi operativi sprovvisti di hardware crittografico dedicato (es. chioschi informativi o comunicazioni *Machine-to-Machine* via API). Ciò richiederà l'addestramento di modelli ML specifici che compensino l'assenza del TPM abbassando drasticamente le soglie di tolleranza e monitorando con maggiore severità l'ancoraggio all'impronta JA3.
3. **Sensor Fusion per il Machine Learning (Integrazione L3/L4):** Attualmente, il *Trust Score* di Splunk MLTK è calcolato su 12 feature derivanti dal livello applicativo (Envoy e Web API). Un potenziamento cruciale consisterà nell'ingestione della telemetria di rete a basso livello all'interno del vettore di addestramento. Trasformare gli allarmi di snort (es. tentativi di scansione Nmap) e i drop log di nftables in feature comportamentali aggiuntive permetterebbe al modello *Gradient Boosting* di correlare anomalie applicative con movimenti laterali di rete, aumentando esponenzialmente la precisione nel rilevamento di minacce persistenti avanzate (APT).
4. **Orchestrazione e Risposta Automatica (SOAR):** L'architettura prevede già un'interfaccia API REST (FastAPI) per l'inserimento dinamico di IP malevoli nella *denylist* di nftables, attualmente utilizzabile dal team SOC per mitigazioni manuali. Questa componente è progettata per evolvere in una pipeline SOAR (*Security Orchestration, Automation, and Response*) completa: istruendo Splunk a lanciare *webhook* automatici verso l'API del firewall non appena il rischio comportamentale supera una soglia critica, si garantirà l'isolamento istantaneo della minaccia a livello kernel (L3) senza alcun intervento umano.
5. **Federated Machine Learning:** In scenari multi-ospedale, il modello *Gradient Boosting* potrebbe evolvere verso paradigmi di *Federated Learning*, permettendo ai SIEM di diverse sedi ospedaliere di addestrare un modello globale condividendo esclusivamente i gradienti di errore e i pesi algoritmici, senza mai scambiarsi i log grezzi contenenti dati clinici e rispettando in modo nativo i vincoli normativi sulla privacy (GDPR).